

算法新平台使用手册

版本：v1.3.0



@ 2020, 版权归凌鸥创芯所有

机密文件，未经许可不可扩散

目录

1.文档版本	1
2.概述	2
2.1 主要特性	2
2.2 demo 文件.....	2
2.2 软件工具	3
2.3 硬件工具	4
2.3.1 MC08x 系列	5
2.3.2 MC03x 系列	5
2.3.3 MC45x 系列	6
2.3.3 MC07x 系列	6
3.算法新平台开发流程	6
3.1 快速开发模式.....	6
3.2 高级开发模式.....	9
3.2.1 模板工程介绍.....	10
3.2.2 用户代码添加.....	11
3.2.3 开发步骤.....	12
3.2.4 各数据层的数据结构.....	14
4.lksmotor 参数配置	16
5.一般调试步骤总结	27
5.1 mcpwm 输出通路测试与硬件过流保护配置.....	28
5.2 开环测试:.....	31
5.3 电流强拖验证:	34
5.3 电流环调试	35
5.4 速度环调试	36
5.5 功率环调试	38
5.6 故障保护功能与恢复	39
5.7 低压压保护异常处理.....	44
5.8 弱磁控制.....	44
5.9 lkscope 常用调试变量	44
5.9.1 电机启停、力矩、转速调节	45

5.9.2 保护标志, 如过流、过压等保护	46
5.9.3 系统状态机查看	47
5.9.4 占空比与采样	48
5.9.5 观测器的输出	49
5.9.6 速度反馈与 PI 调节	49
5.9.7 电流反馈与 PI 调节	49
5.9.8 算法状态变量	49
5.9.9 电机控制参数(即 lksmotor 配置的参数)	50
6. 常见异常处理	51
6.1 错误保护标志	51
6.2 启动电流大	51
6.3 明显抖动	52
6.4 停机检测失败	52
6.5 偏置电压读取	53
6.6 电机启动运行进入伪收敛状态	53
7. 不同硬件之间软件移植	54
7.1 硬件说明	54
7.2 一般修改参数模块:	55
7.3 驱动极性分析	55
8 数据格式	59
8.1 标幺定义	59
8.2 上位机数据单位说明	63
9. 新平台控制状态机	63
9.1 UA 层状态机	63
9.2 MA 层状态机	65
10. 补充说明	68
版本历史	70

1.文档版本

本文档是针对无感 **foc** 无代码开发平台的使用说明手册，用户可以参考该手册了解相关参数的具体配置细节和调试开发流程，实际的软件若有迭代更新，以实际的软件为主。

文件说明	算法新平台使用说明
日期	2024.09.28
适用 MCU	LKS32MC03x & LKS32MC08x & LKS32MC07x & LK32SMC45x
控制模式	电流/速度/功率环路
采样模式	单/双/三电阻
有感/无感	无感
调试工具	lkscope1.2.8 版本
编制	ywx、wzz

表 1-1：新平台信息说明

2.概述

2.1 主要特性

新算法软件平台是凌鸥提供的针对直流无刷电机,永磁同步电机进行无感 foc 控制开发的图形化配置平台,以 lksmotor 为主,结合相关编译工具(keil)和调试工具(lksscope)及其工程例程,可以很好上手开发电机控制相关项目,可以基本适配以下功能:

- 顺逆风启动
- SLVC 启动
- 初始位置检测
- 电流环、速度环、功率环控制
- 单、双、三电阻拓扑结构采样
- 硬/软件过流、过压保护机制

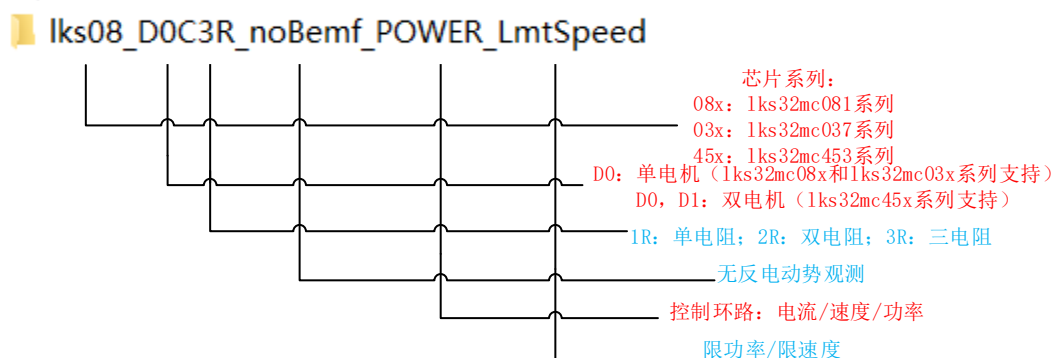
2.2 demo 文件

新算法平台提供的 lksmotor 软件包中, 针对不同系列的 mcu 提供了基本的测试 demo 和 lksscope 调试配置工程, 具体的 demo 在提供的 ceshi 文件夹下:

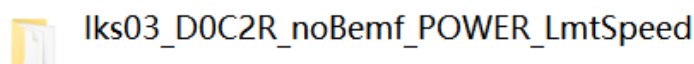
新平台推广 > 新平台软件 > Lksmotor_V1.0.9_AC5 > ceshi > AC5 > 03x > V2_0_9_LKS10			
名称	修改日期	类型	
lks03_D0C1R_noBemf_CURRENT	2023/11/29 15:48	文件夹	
lks03_D0C1R_noBemf_POWER	2023/11/29 15:48	文件夹	
lks03_D0C1R_noBemf_POWER_LmtSpeed	2023/11/29 15:48	文件夹	
lks03_D0C1R_noBemf_SPEED	2023/11/30 9:28	文件夹	
lks03_D0C1R_noBemf_SPEED_LmtPower	2023/11/29 15:48	文件夹	
lks03_D0C2R_noBemf_CURRENT	2023/11/29 15:48	文件夹	
lks03_D0C2R_noBemf_POWER	2023/11/29 15:48	文件夹	
lks03_D0C2R_noBemf_POWER_LmtSpeed	2023/11/29 15:48	文件夹	
lks03_D0C2R_noBemf_SPEED	2023/11/29 15:48	文件夹	
lks03_D0C2R_noBemf_SPEED_LmtPower	2023/11/29 15:48	文件夹	

图 2-1 测试 demo 文件

其中文件夹的命名规则如下:



例如下面的测试文件代表 LKS32MC037x 系列+单电机双电阻+无反电动势检测+功率环+限速度测试 demo。



2.2 软件工具

系统软件由 2 个部分组成：**KEIL 工程+lksmotor 参数配置**

- 下位机 KEIL 工程：算法控制-无感 FOC 算法/硬件初始化/保护判断等
- 上位机配置 lksmotor：参数配置-MCU 系列选择/电流采样/母线电压采样参数配置等
- 上位机调试软件 lksscope：用于无感 foc 相关变量的在线调试工具



图 2-2：新平台软件组成

下位机 KEIL 工程分为 LKS32MC081x、LKS32MC037x、LKS32MC453 两个 MCU 系列模板程序，其中包含针对两个编译器版本的工程，用户可以根据编译需求选择不同的代码模板

名称	修改日期	类型
1、LKS32MC03x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
1、LKS32MC07x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
1、LKS32MC08x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
LKS32MC45x_2Motor2R_1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹

图 2-3：下位机 KEIL 工程

lksmotor 通过配置属性来选择 LKS32MC08x、LKS32MC03x 或者 LKS32MC45x 系列不同

控制方式的固件名称，用户可以根据自己的需求进行选择，具体固件的名称命名规则如下：

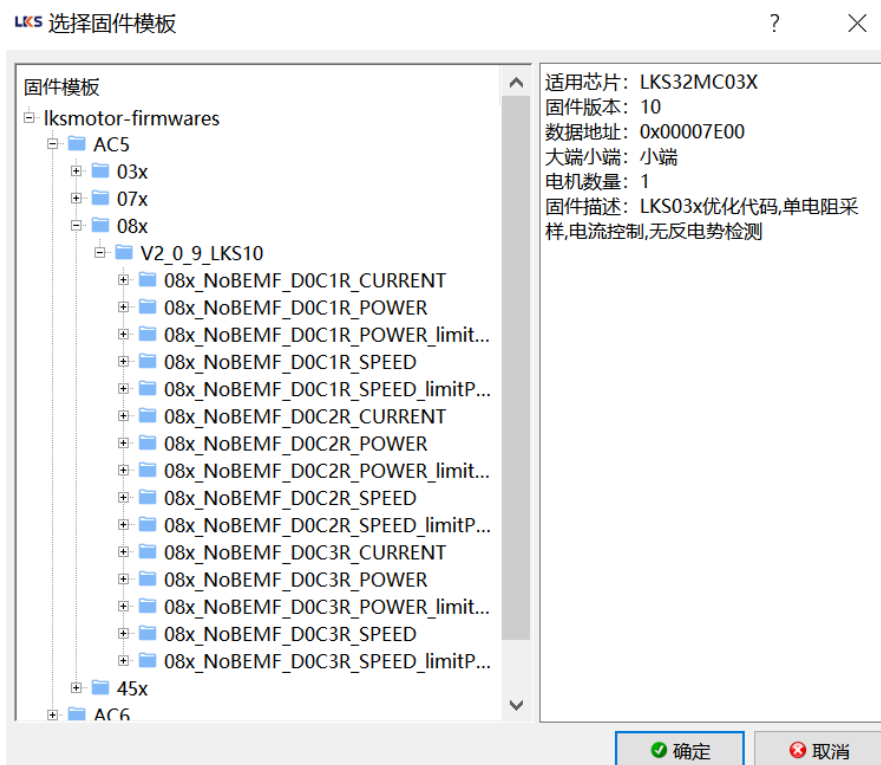
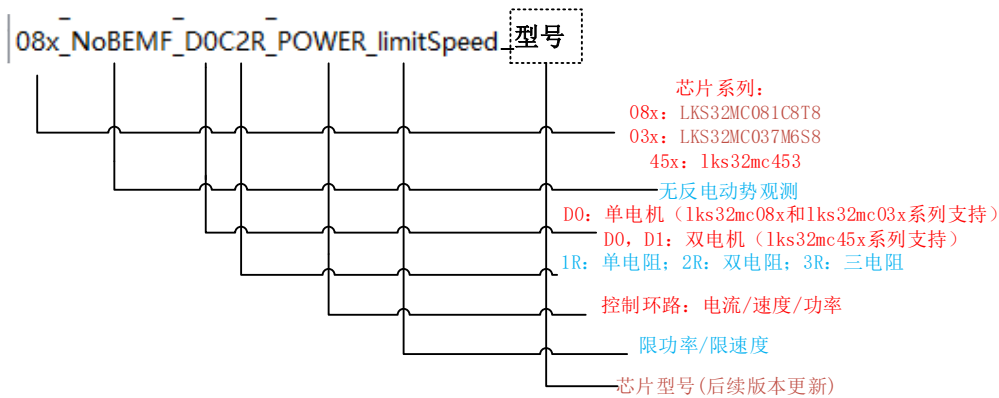


图 2-4:固件选择

2.3 硬件工具

凌鸥针对不同的 MCU 系统都提供了对应的 mcu 板和功率应用板，有关详细信息，请参见凌鸥官方网站 <https://www.lksmcu.com/index.php/DemoBoard/>

软件平台支持 lksmcu 的不同系列、其中提供的基础 08x、03x、45x 模板例程分别对应 lks32mc081 系列，03x 对应 lks32mc037，45x 对应 lks32mc453 系列，具体相关硬件配套系列相关示例如下：

2.3.1 MC08x 系列

示例工程代码使用 Demo 硬件为 LKS32MC081C8T8 + 外置预驱 LKS560 功率板

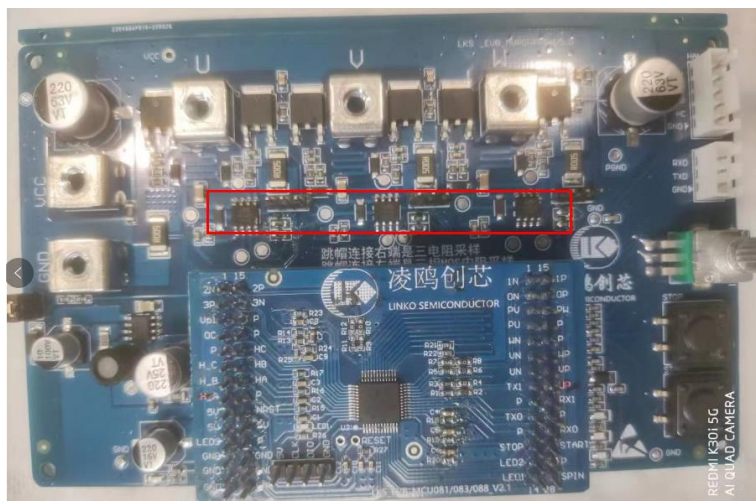


图 2-5: 示例工程使用的 MC08x 硬件

如果采用 MC08x 其它型号,请对照 [LKS32MC08X Datasheet](#), 修改对应的 MCPWM 模块配置, 可参考第八部分移植说明

2.3.2 MC03x 系列

示例工程代码使用 Demo 硬件为 LKS32MC037M6S8 + 外置预驱 LKS560 功率板, 如果采用 MC08x 其它型号,请对照 [LKS32MC08X Datasheet](#), 修改对应的 MCPWM 模块配置, 可参考第八部分移植说明

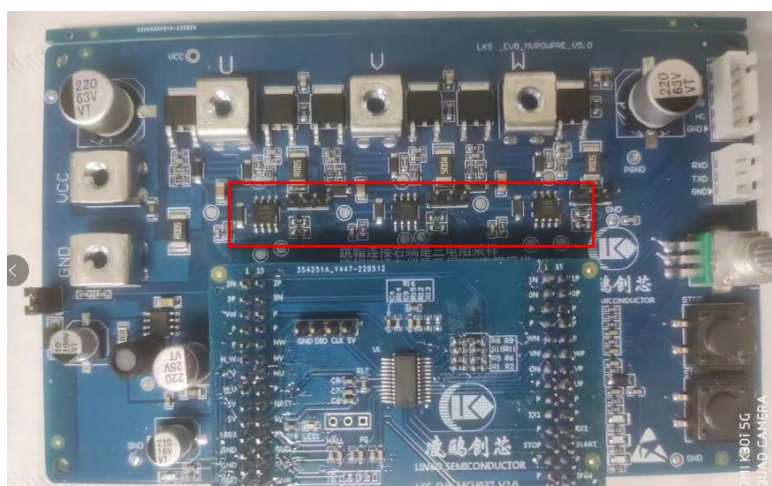


图 2-6: 示例工程使用的 MC03x 硬件

如果采用 MC03x 其它型号,请对照 [LKS32MC03X Datasheet](#), 修改对应的 MCPWM 模块配置, 可参考第八部分移植说明

2.3.3 MC45x 系列

示例工程代码使用 Demo 硬件为 LKS32MC453RCT8 + 外置预驱 LKS560 功率板，如果采用 MC45x 其它型号,请对照 [LKS32MC45X Datasheet](#)，修改对应的 MCPWM 模块配置，可参考第八部分移植说明

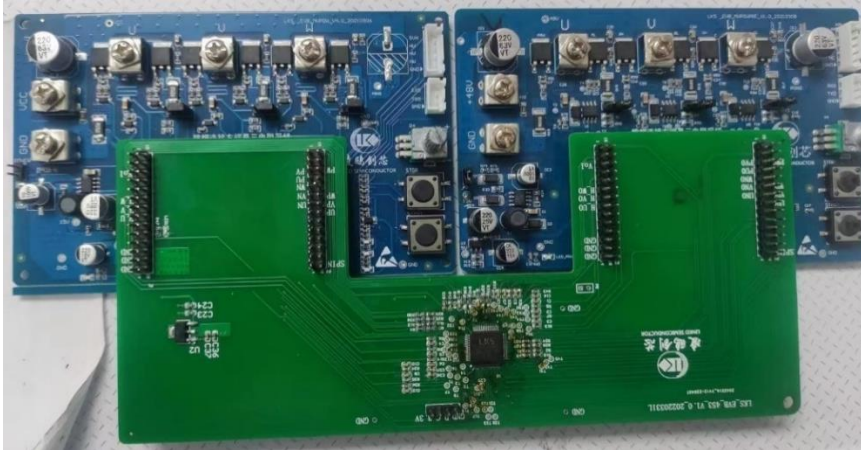


图 2-7：示例工程使用的 MC45x 硬件

2.3.3 MC07x 系列

示例工程代码使用 Demo 硬件为 LKS32MC071C8T8 + 外置预驱 LKS560 功率板，如果采用 MC07x 其它型号,请对照 [LKS32MC07X Datasheet](#)，修改对应的 MCPWM 模块配置，可参考第七部分移植

3.算法新平台开发流程

新算法平台可以让用户根据自己的电机控制项目中的需求，进行无感 foc 图形化开发，具体可以分为快速开发模式和高级开发模式两种，下面将从两种模式进行具体的介绍：

3.1 快速开发模式

用户可以使用凌鸥官方提供的对应 mcu demo 板和功率底板，仅仅使用 lksmotor 软件就快速上手开发驱动自己的电机，具体的开发流程如下：

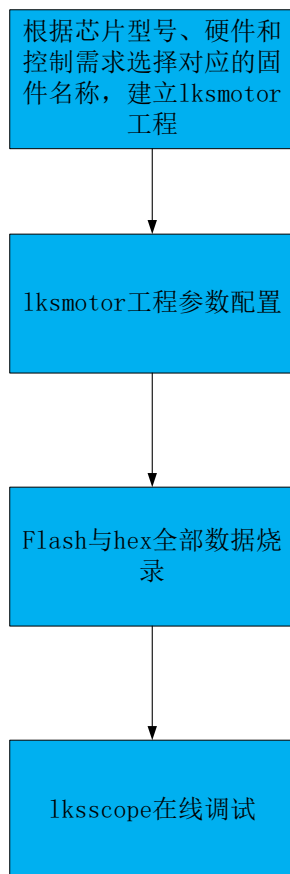


图 3.1 快速开发流程

步骤 1: 打开 lksmotor 软件，用户可以新建自己的工程文件或者打开官方提供的工程（官方提供的测试工程在 ceshi 文件夹下），新建工程的步骤如图所示，其中固件的选择，用户可以根据自己项目的 mcu 系列和需要的控制方式进行选择，固件名称的命名规则同提供的 demo 文件命名规则相同，详见 [2.2 固件选择](#)



图 3.2 新建工程

步骤二：参数配置，针对自己电机的相关参数和功率板的硬件参数，进行配置，具体 lks

motor 相关参数的配置和含义，请参考第四章 lksmotor 参数配置。参数配置完成以后，用户应点击保存配置，会在当前工程路径下自动生成 lkscope 调试工程文件，方便用户在线调试

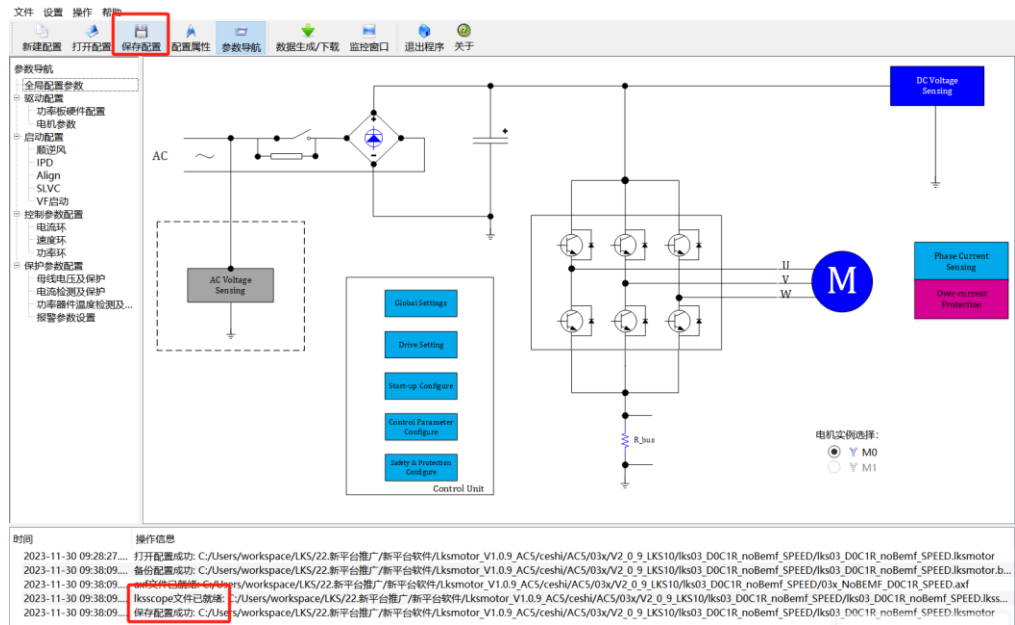


图 3-3 生成调试工程

步骤三：下载烧录，烧录方式选择 flash+hex，烧写时钟尽量选择在 10Mhz 以下，烧录成功会有进度条提示

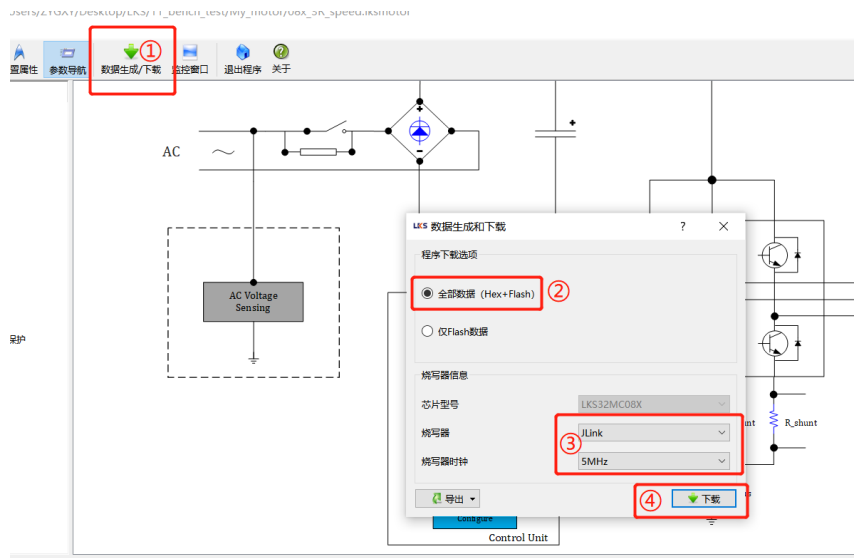


图 3-4 下载步骤

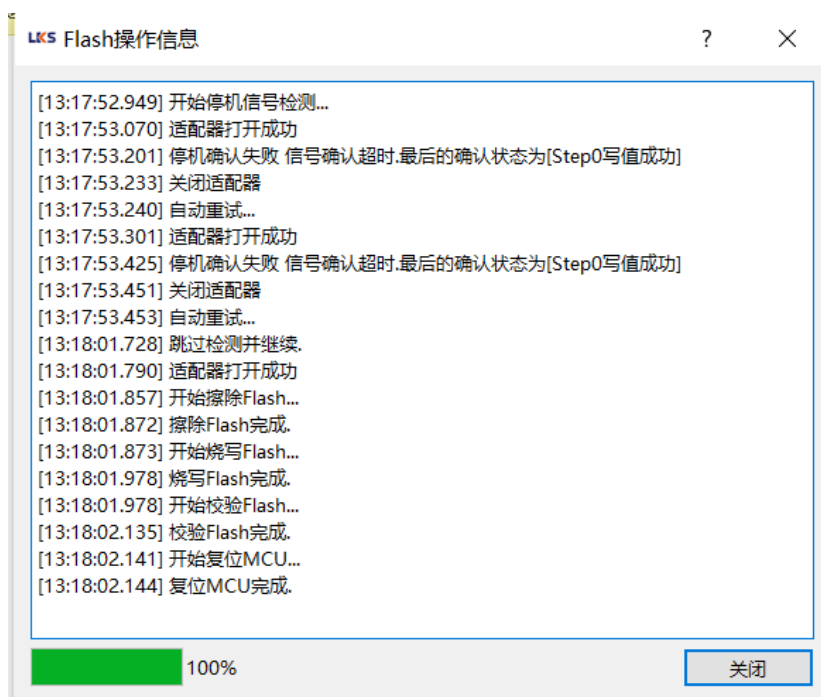


图 3-5 烧录成功

步骤四：在线调试，用户打开工程的存放路径下的 **lksscope** 工程，对应 **lksscope** 已经将常用的调试变量配置完成，不需要用户自己在添加，用户可以根据实际的调试需求，查看其他变量，对于常用的调试变量，请参考第 5 章在线调试中的相关说明。

3.2 高级开发模式

凌鸥官方针对不同系列 MCU 提供了对应的模板例程，对于需要相关底层开发需求的用户，可以结合模板例程进行代码的开发，利用 **lksmotor** 平配置相关参数，同时结合 **lksscope** 进行调试。结合凌鸥官方提供的配套工具，具体的开发流程如图 3-1

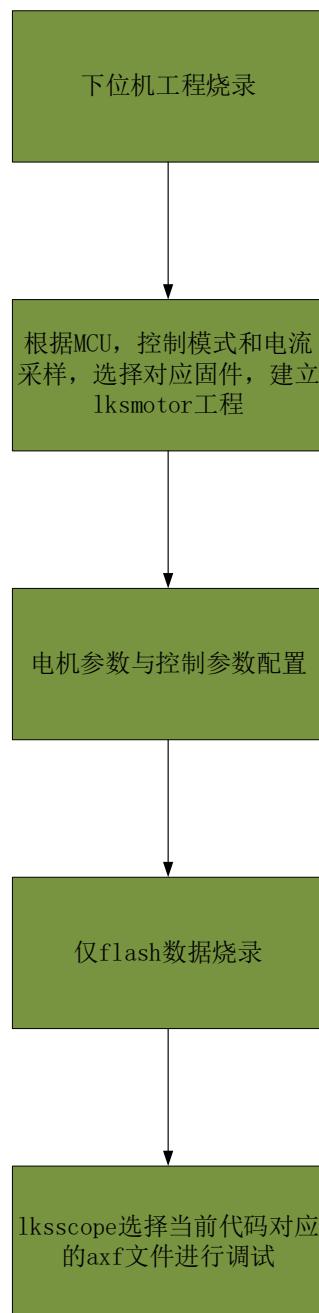


图 3-5 高级开发流程

3.2.1 模板工程介绍

算法新平台支持 LKS32MC08x/03x/45x 全系列 mcu 无感 foc 图形化开发，提供了三个 mcu 系列的基本模板工程，配合上位机 lksmotor 进行开发，具体工程框架的介绍如下

工程框架如图 3-6: MCU 外设驱动、硬件驱动层、【公共层、电机驱动层、电机应用层】、用户应用层、参数配置；

用户只需修改**硬件驱动层、用户层、参数配置**这 3 个部分即可完成产品的设计，其中

【公共层、电机驱动层、电机应用层】是算法的实现过程，为了减轻应用的负担，用户可以不用了解，*建议用户不要修改这3层的代码以防出现控制错误*

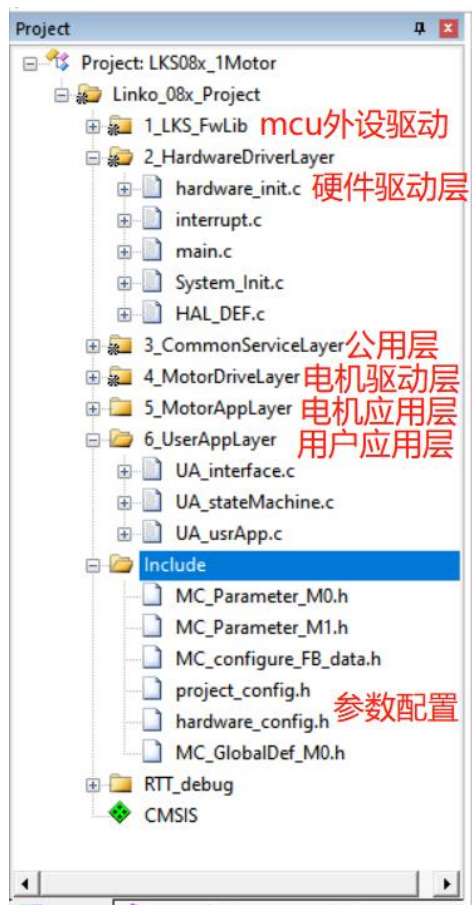


图 3-6：下位机 KEIL 工程架构

3.2.2 用户代码添加

建议在用户应用层 UserAppLayer 来添加自己的功能代码，在该层中有 3 的文件：UA_stateMachine.c 是低频任务状态机，方便用户了解电机控制过程；UA_interface.c 用户应用层到电机应用层的接口；用户添加功能在 USER_APP.c 中，这里代码调用是在低频任务中调用的，调用周期 10ms,具体软件更新，相关任务执行频率以最新的软件工程为主。用户增加逻辑功能建议在 USER_APP.c 的 updateM0DebugUA 函数中

这里简单示例一下添加功能代码，一般用户代码都和电机状态相关的，不同状态执行不同功能代码功能：检测电机运行状态，当运行状态 = 运行时，调试灯开，其它状态调试灯关

```
void updateM0DebugUA(pSTR_UACtrProc pObj)
{
/*****定时自动启停功能*****/
if (runM0UaCmdAuto)
{
    runM0Period++;
    if (runM0Period >= 10000)
```

```

    {
        if(runM0UaCmd == FALSE)
        {
            runM0UaCmd = TRUE;
        }
        else
        {
            runM0UaCmd = FALSE;
        }
        runM0Period = 0;
    }
}
else
{
    runM0Period = 0;
}
/*****定时自动启停功能*****/

pObj->m_wSetpoint                = runM0SpdSetpoint; //转速设定值

if(runM0UaCmd)                   //UA 层启动控制标志置 1
{
    pObj->m_StruCmnd.m_bMoeEnable = 1;
    pObj->m_StruCmnd.m_bRunFlag   = 1;
}
else                             //UA 层启动控制标志置 0
{
    pObj->m_StruCmnd.m_bMoeEnable = 0;
    pObj->m_StruCmnd.m_bRunFlag   = 0;
}

if(pObj->m_StruCmnd.m_bDownLoadStart == 1) //程序下载期间运行标志清 0
{
    pObj->m_StruCmnd.m_bMoeEnable = 0;
    pObj->m_StruCmnd.m_bRunFlag   = 0;
}
if(pObj->m_eUASysState == E_UA_RUN) //电机运行指示灯开
{
    LED_ON();
}
else
{
    LED_OFF();
}
}
}

```

图 3-7：用户功能代码示例

3.2.3 开发步骤

步骤一：下位机程序烧录，这里烧录的方式有两种方法：

- 方法 1：使用 KEIL 工程下载程序、打开工程图标

名称	修改日期	类型	大小
1_LKS_FwLib	2023/5/8 14:19	文件夹	
2_HardwareDriverLayer	2023/5/8 14:19	文件夹	
3_CommonServiceLayer	2023/5/8 14:19	文件夹	
4_MotorDriveLayer	2023/5/8 14:19	文件夹	
5_MotorAppLayer	2023/5/8 14:19	文件夹	
6_UserAppLayer	2023/5/8 14:19	文件夹	
Include	2023/5/8 14:19	文件夹	
Listings	2023/5/8 14:19	文件夹	
Objects	2023/5/8 14:19	文件夹	
RTE	2023/5/8 14:19	文件夹	
RTT	2023/5/8 14:19	文件夹	
EventRecorderStub.scvd	2022/9/30 10:47	SCVD 文件	1 KB
JLinkLog.txt	2023/4/28 16:47	文本文件	163 KB
JLinkSettings.ini	2020/8/16 20:07	INI 源文件	1 KB
LKS_08x.lksviewer	2023/4/28 16:48	LKSVIEWER 文件	27 KB
LKS08x_1Motor.uvguix.bpsemi	2023/4/28 16:48	BPSEMI 文件	195 KB
LKS08x_1Motor.uvoptx	2023/4/20 16:21	UVOPTX 文件	30 KB
LKS08x_1Motor.uvprojx	2023/4/20 16:21	Microvision5 Project	37 KB
mapinit.ini	2016/2/22 14:43	INI 源文件	1 KB
Project.pmp	2023/2/20 13:49	PMP 文件	62 KB
project_clean.bat	2022/5/23 14:55	Windows 批处理...	1 KB

图 3-8：下位机 MC08x KEIL 工程

编译后点击 download 图标



图 3-9：KEIL 烧录程序

• 方法 2：使用 LKSFLASH 下载程序，具体步骤如下：

<1>配置基本参数，特别注意 MCU 型号选取对应没有问题

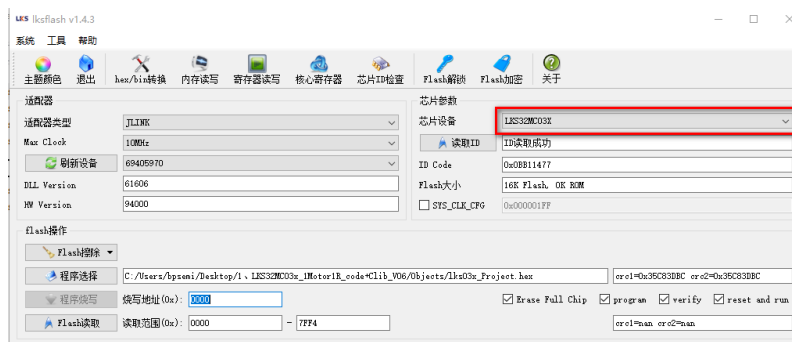


图 3-10：LSKFLASH 选择 MCU 系列

<2>程序选择、烧写并等待完成

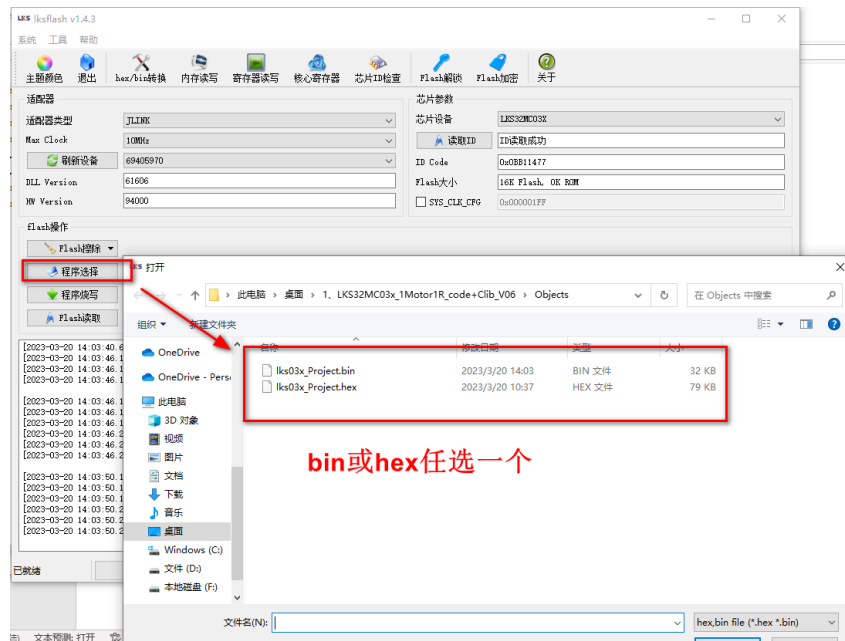


图 3-11: LSKFLASH 选择烧录文件

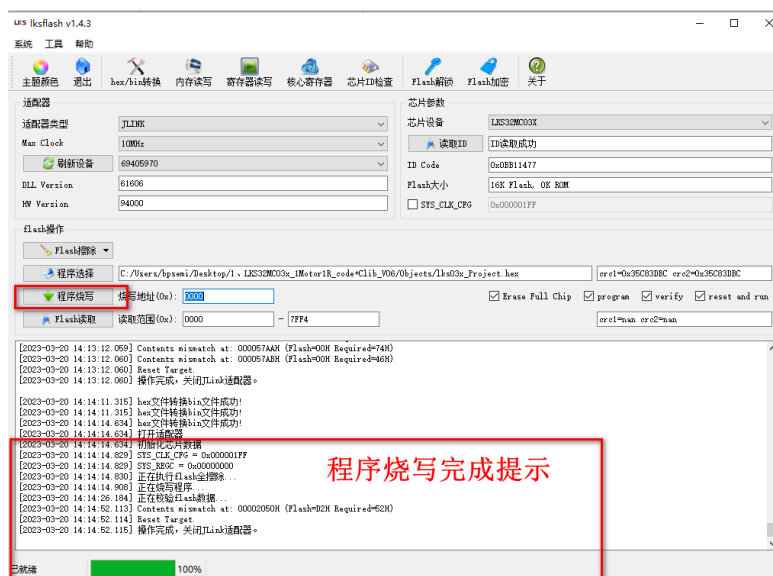


图 3-12: 程序烧录完成

步骤三：参数配置，用户烧录完成下位机程序以后，打开 lksmotor，建立对电机的相关参数和控制方法的工程，具体 lksmotor 的参数配置可以参考第 4 章的具体介绍，配置完成以后保存配置工程，lksmotor 在对应的工程路径下生成 lksscope 调试工程，方便用户进一步调试。

步骤四：在线调试：打开对应工程路径下的 lksscope，结合具体项目要求进行调试，常用的调试变量已经配置完成，具体可见第 5 章 lksscope 常用调试变量。

3.2.4 各数据层的数据结构

当前的代码分为用户层、电机应用层、电机控制层。针对当前程序中采用的数据，用户

层数据为 User Data，电机应用层数据为 App Data，电机控制层为 Core Data。用户层数据即 User Data 采用 float 数据时，代码编译后会引入浮点库，占用较大的 FLASH 空间。因此，03x 和 08x 的程序模板默认用户层采用 App Data 的数据格式进行处理。所有的配置参数对外显示表示为 float 数据，实际程序中应用时通过宏定义或者通过 lksmotor 转换为定点数进行处理。

参数配置数据通过宏定义的方式，下载到指定 flash 的指定地址中，在查找某一个参数配置的头文件在程序中哪里用到的时候，之间搜索定义的宏名称是找不到对应的变量的，搜索定义参数在代码中的位置时，应该把后缀的 M0 的 0 去掉进行搜索，如下图进行全局搜索，便可以定位到定义参数赋给的变量的位置，图 3.15 是在 keil 下全局搜索对应的宏 U_OVERVOLTAGE_RECOVER_M0，图 3.16 可以定位到在 MC_configure_FB_data.h 中的 613 行，绿框中的 600 行的变量正好与红框中的对应，从而找对对应的变量在第 600 行。

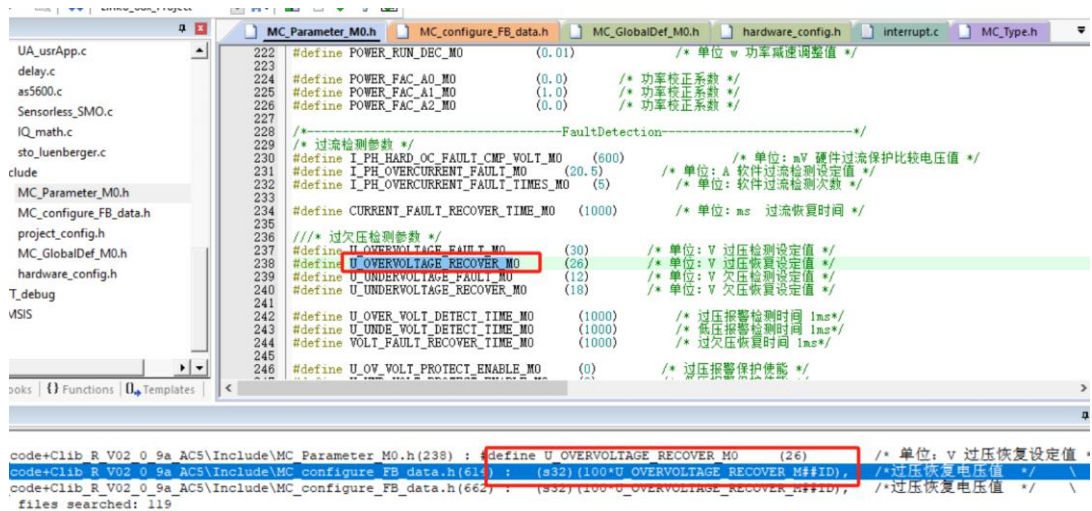


图 3.13 搜索某一个宏参数定义

```
typedef struct
{
    s32      m_WDCoverVoltage;           // 过压保护阈值      1
    s32      m_WDCoverVoltageRecover;    // 过压恢复电压值    1
    s32      m_WDCUnderVoltage;          // 低压保护阈值      1
    s32      m_WDCUnderVoltageRecover;    // 低压消除恢复电压值 1
    u16      nOvVoltageDetectTime;        // 过压检测时间      1
    u16      nUndVoltageDetectTime;        // 欠压检测时间      1
    u16      nVoltageRecoverTime;          // 电压恢复正常等待时间 1
    s8       bOvVoltProtectEnbale;        // 使能过压保护      1
    s8       bUndVoltProtectEnbale;        // 使能低压保护      1
}STR_FB_DC_UNIT0;

#define DC_UNIT0_PARAMETER_TAB_M(ID)
{
    (s32)(100*U_OVERVOLTAGE_FAULT_M##ID), //过压保护阈值      */
    (s32)(100*U_OVERVOLTAGE_RECOVER_M##ID), //过压恢复电压值    */

```

```

(s32) (100*U_UNDERVOLTAGE_FAULT_M##ID), /*低压保护阈值 */ \
(s32) (100*U_UNDERVOLTAGE_RECOVER_M##ID), /*低压消除恢复电压值*/ \
(u16) U_OVER_VOLT_DETECT_TIME_M##ID, /*过压检测时间 */ \
(u16) U_UNDE_VOLT_DETECT_TIME_M##ID, /*欠压检测时间 */ \
(u16) VOLT_FAULT_RECOVER_TIME_M##ID, /*电压恢复正常等待时间*/ \
(u8) U_OV_VOLT_PROTECT_ENABLE_M##ID, /*使能过压保护 */ \
(u8) U_UND_VOLT_PROTECT_ENABLE_M##ID /*使能低压保护 */ \
}

```

图 3.14 定位宏定义参数赋给的变量

4.lksmotor 参数配置

lksmotor 是凌鸥针对无感 foc 开发推出的图形化配置软件，可以根据电机类型以及硬件的需要，进行图形化参数配置，进而高效的进行电机控制项目的开发，下面从 lksmotor 的具体参数以及相关配置流程进行具体的介绍：

<1>找到软件的路径，打开 lksmotor，新建工程

名称	修改日期	类型	大小
ceshi	2023/11/29 15:48	文件夹	
lks-data	2023/11/29 15:48	文件夹	
lksmotor-firmwares	2023/11/29 15:48	文件夹	
LKS lksmotor_v1.1.1.exe	2023/11/27 9:34	应用程序	23,150 KB

图 4-1：打开 lksmotor

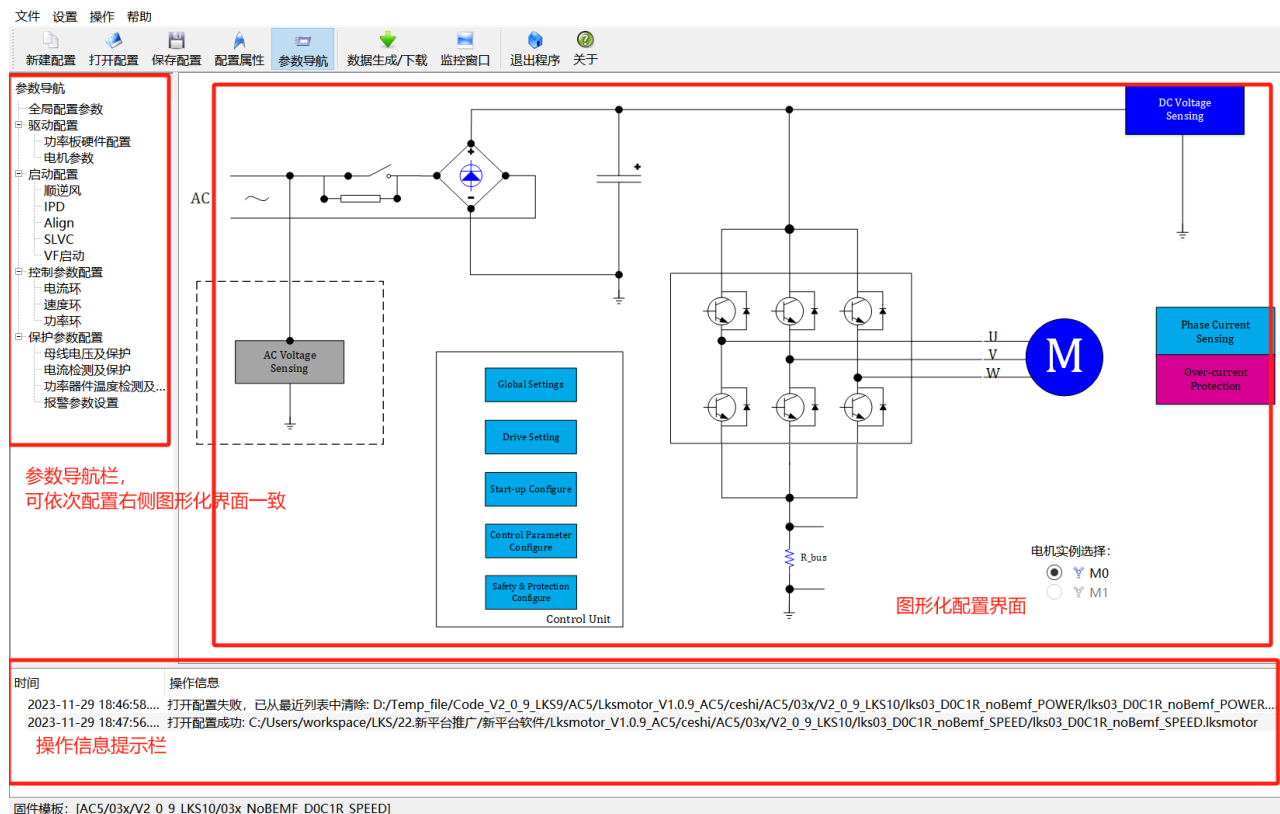


图 4-2: lksmotor 界面

<2>参数配置界面说明

用户可以单击工具栏上的参数导航选项，查看具体的参数配置项，参数导航详细的罗列了可配置的参数项，与图形化配置界面一一对应图 4-3，用户可以在图形化界面单击图标进行配置，也可以选择参数导航界面进行配置，二者的配置效果是一样的。

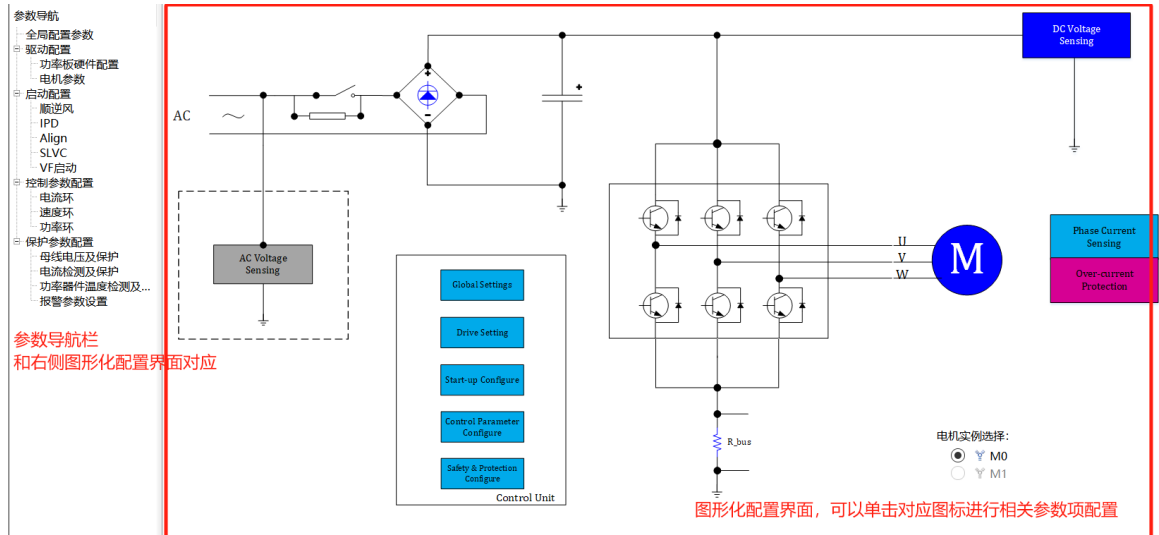


图 4-3: 参数配置界面

<3>参数权限说明

灰色：不可修改，用户在建立工程选择固件的时候已经默认配置完成；亦或只能通过下位机修改的选项(这个部分在下面会列出)，举例如下图 4-4

全局配置参数 电机实例-M0

参数名称	参数描述	数值	数值类型	量纲
MCU_MCLK_M0	MCU主频时钟	96	s16	MHz
PWM_MCLK_M0	PWM模块时钟	96	s16	MHz
PWM_FREQ_M0	PWM载频率	16000	u16	Hz
PWM_PRSC_M0	PWM预分频	0	u16	1
SPD_LOOP_FREQUENCY_M0	速度环执行频率	1000	u16	Hz
DEADTIME_NS_M0	死区时间	1200	u16	1ns
CURRENT_SAMPLE_TYPE_M0	电流采样方式	3-三电阻	u16	1
SVPWM_TYPE_M0	SVPWM方式	0-七段式	u8	1
PWM_USED_ID_M0	选用的PWM及AD端口	0-EPWM0	u8	1
OPA_SELECT_M0	选用的OPA放大选项	0-200/10.4	u8	1

根据固件选择情况，配置已经默认，不可修改

确定 取消

图 4-4：不可修改参数权限说明

黑色：可以修改，直接输入或下拉列表形式，举例如下图 4-5

参数名称	参数描述	数值	数值类型	单位
MCU_MCLK_M0	MCU主频时钟	48	s16	MHz
PWM_MCLK_M0	PWM模块时钟	48	s16	MHz
PWM_FREQ_M0	PWM载频率	16000	u16	Hz
PWM_PRSC_M0	PWM预分频	0	u16	1
SPD_LOOP_FREQUENCY_M0	速度环执行频率	1000	u16	Hz
DEADTIME_NS_M0	死区时间	800	u16	1ns
CURRENT_SAMPLE_TYPE_M0	电流采样方式	2-双电阻	u16	1
SVPWM_TYPE_M0	SVPWM方式	0-七段式	u8	1
PWM_USED_ID_M0	选用的PWM及AD端口	0-EPWM0	u8	1
OPA_SELECT_M0	选用的OPA放大选项	0-200/10 0-200/10 1-190/20 2-180/30 3-170/40	u8	1

图 4-5：可修改参数权限说明

<4>配置参数说明

按照图 4-6 中 15 个部分进行参数说明（注：因为软件版本迭代相关配置参数项有更新的情况，用户以软件为主），本文档是对基本相关参数的介绍，用户有其他的疑惑请联系凌鸥官方技术人员。

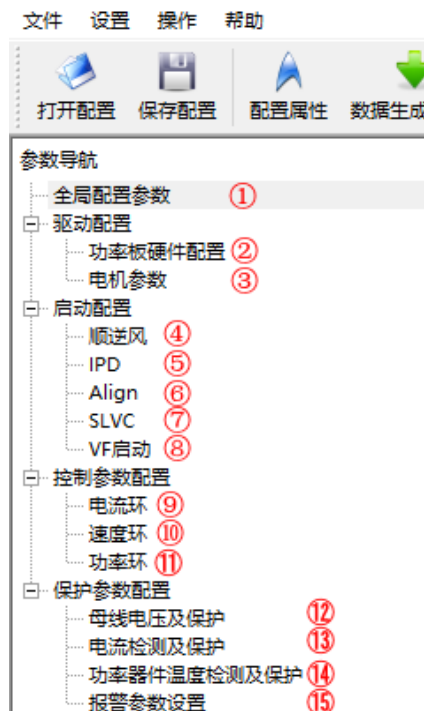


图 4-6：lksmotor 配置目录

lksmotor 配置参数有如图 4-6 的 15 个部分，其中⑧在⑦中已经实现，只介绍⑦，不再说明⑧；

⑭是温度检测保护的参数设定，实际中大部分电机没有此功能，这里暂未实现；

⑫、⑬部分和硬件紧密相关，在②功率板硬件配置中会涉及二者配置，就不单独列出阐述；

用户根据自己的实际硬件选择模块参数配置，比如有的产品没有使用到反电动势的相关功能，可以不进行配置；

以下对关键参数进行说明，对于‘见名知意’的参数不再赘述，如果遇到疑问的地方可与技术人员联系；

①

全局配置参数 电机实例-M0

硬件配置 功能配置 参数配置

参数名称	参数描述	数值	数值类型	量纲
MCU_MCLK_M0	MCU主频时钟	48	integer	1MHz
PWM_MCLK_M0	PWM模块时钟	48	integer	1MHz
PWM_FREQ_M0	PWM载频率	16000	integer	1Hz
PWM_PRSC_M0	PWM预分频	0	integer	1
SPD_LOOP_FREQUENCY_M0	速度环执行频率	1000	integer	1Hz
OVER_MODU_FACTOR_M0	电压调制系数	18919	integer	1
DEADTIME_NS_M0	死区时间	1200	integer	1ns
CURRENT_SAMPLE_TYPE_M0	电流采样方式	1-单电阻	integer	1
SVPWM_TYPE_M0	SVPWM方式	0-七段式	integer	1
PWM_USED_ID_M0	选用的PWM及AD端口	0-EPWM0	integer	1
OPA_SELECT_M0	选用的OPA放大选项	0-200/10	integer	1

预分频一般设置为0
死区时间根据硬件调整
放大倍数根据实际需要选择

硬件配置 **功能配置** 参数配置

参数名称	参数描述	数值	数值类型	量纲
DIR_EMF_CHECK_ENABLE_M0	使能反电势顺逆风检测	0-Disable	integer	1
CHARGE_STATUS_M0	使能预充电	1-Enable	integer	1
SEEK_POSITION_STATUS_M0	使能IPD	1-Enable	integer	1
DIR_CHECK_STATUS_M0	使能顺逆风检测	1-Enable	integer	1
ALIGN_STATUS_M0	使能对相	1-Enable	integer	1
VF_START_STATUS_M0	使能VF启动	0-Disable	integer	1
CLOSE_LOOP_M0	控制模式	1-速度环	integer	1
POWER_LIMIT_STATUS_M0	速度环 限制功率	0-Disable	integer	1
SPPED_LIMIT_STATUS_M0	功率环 限制速度	0-Disable	integer	1
WEAKEN_FLUX_STATUS_M0	弱磁控制使能	1-Enable	integer	1
PWM_ENABLE_STOP_M0	停止时PWM使能	0-Disable	integer	1
FAULT_AUTO_CLR_M0	故障自动清除使能	1-Enable	integer	1
STOP_MODE_M0	停止方式	0-关闭MOE	integer	1

是否使能IPD
是否使能顺逆风
是否使能对相
环路用户也可以修改下位机程
前期调试阶段可以先DISABLE

硬件配置 功能配置 **参数配置**

参数名称	参数描述	数值	数值类型	量纲
CALIB_SAMPLES_M0	AD校正次数	512	integer	1次
ADC_OFFSET_ERROR_M0	AD校正允许偏差	3500	integer	1
CHARGE_TIME_M0	充电时间	30	integer	1ms
FAULT_CLR_PRIOD_M0	故障清除周期	2000	integer	1

电机运行前计算电流偏差误差允许范围
上mos是N管的时候自举电容充电时间
故障清除周期和上面自动清楚使能配合使用

图 4-7：全局参数配置



图 4-8：硬件采样

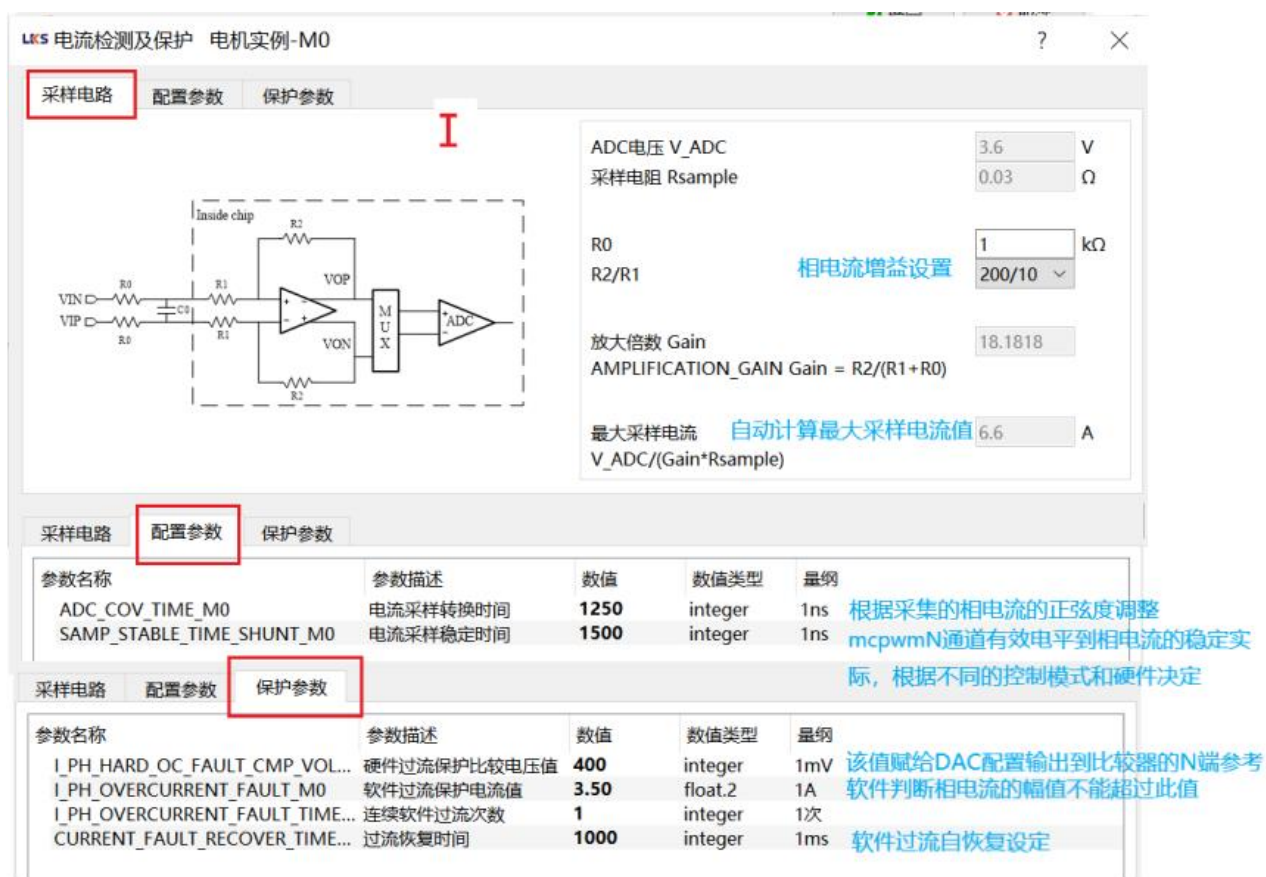


图 4-9：电流采样

采样电路

配置参数

II

ADC电压 V_ADC

3.6 V

R1

40 kohm

R2

1 kohm

由实际硬件决定

母线电压采样分压比

0.0243902

VOLTAGE_SHUNT_RATIO = R2/(R1+R2)

最大测量范围

147.6

V_ADC/VOLAGE_SHUNT_RATIO

采样电路

配置参数

参数名称	参数描述	数值	数值类型	量纲
U_OVERVOLTAGE_FAULT_M0	过压保护阈值	30.00	float.2	1V
U_OVERVOLTAGE_RECOVER_M0	过压恢复电压值	26.00	float.2	1V
U_UNDERVOLTAGE_FAULT_M0	低压保护阈值	12.00	float.2	1V
U_UNDERVOLTAGE_RECOVER_M0	低压消除恢复电压值	18.00	float.2	1V
U_OVER_VOLT_DETECT_TIME_M00	过压检测时间	1000	integer	1ms
U_UNDE_VOLT_DETECT_TIME_M0	欠压检测时间	1000	integer	1ms
VOLT_FAULT_RECOVER_TIME_M0	电压恢复正常等待时间	1000	integer	1ms
U_OV_VOLT_PROTECT_ENABLE_M0	使能过压保护	0-Disable	bool	
U_UND_VOLT_PROTECT_ENABLE_...	使能低压保护	0-Disable	bool	

电压保护检测周期

图 4-10：母线电压采样

采样电路

配置参数

III

ADC电压 V_ADC

3.6 V

R1

40 kohm

R2

1 kohm

反电势电压采样分压比

0.0243902

VOLTAGE_SHUNT_RATIO = R2/(R1+R2)

最大测量范围

147.6

V_ADC/VOLAGE_SHUNT_RATIO

图 4-11：反电动势采样

参数名称	参数描述	数值	数值类型	量纲
U_MOTOR_PP_M0	极对数	5	integer	1对
U_MAX_FREQ_M0	额定频率	1333.00	float.2	1Hz
U_MOTOR_RS_M0	电机相电阻 (25°C)	0.4443	float.4	1 ohm
U_MOTOR_LD_M0	电机D轴电感	234	integer	1uH
U_MOTOR_LQ_M0	电机Q轴电感	344	integer	1uH
U_MOTOR_FLUX_CONST_M0	电机磁链常数	0.01747	float.5	1WB

图 4-12：电机参数设定

④

LKS 顺逆风 电机实例-M0

参数名称	参数描述	数值	数值类型	量纲
SPEED_TRACK_DELAYTIME_M0	顺逆风检测时间	500	s16	ms
SPEED_TRACK_ON_FREQ_THH_M0	顺风切闭环频率	25.00	float	1Hz
EBRK_ON_FREQ_THH_M0	逆风刹车频率	25.00	float	1Hz
MOTOR_STOP_CUR_THD_M0	电机停转电流检测阈值	0.20	float	A
MOTOR_STOP_CUR_DIF_THD_M0	电机停转电流差值检测阈值	0.10	float	A
STOP_TIME_M0	电机停止检测滤波时间	200	s16	ms
STOP_DELAY_TIME_M0	电机停止后延迟时间	200	s16	ms
BEMF_ANG_COMP_M0	反电势角度补偿	0	s16	度
BEMF_TRACK_MIN_VOLT_M0	反电势检测最小电压	200	s16	0.01V

程序检测到电机停止（实际可能还在低速转动），需要在延迟一段，以方便新的启动

连续这个时间才算是停止

电流峰值

计算峰值变化的差值

根据实际检测的电流判断顺逆风，有时候两个条件一起限制

☒ 使能顺逆风检测

确定 取消

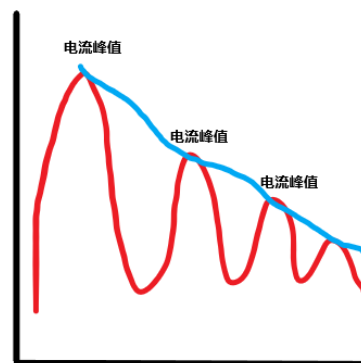


图 4-13：顺逆风参数配置

⑤

LKS IPD 电机实例-0

参数名称	参数描述	数值	数值类型	量纲
U_START_ANGLE_COMP_M0	初始位置检测补偿角度	0.00	float	度
IPD_PLUS_TIME_SETTING_M0	脉冲注入时间宽度设置	50	u16	1us
IPD_WAIT_TIME_SETTING_M0	脉冲空闲等待时间宽度设置	800	u16	1us

默认认为0

输入时间宽度越大，注入电流就越大，识别初始位置相对准确一点，但会产生噪声

注入脉冲电流之间的时间间隔

这两个参数效果，可以使用示波器+电流钳测量

图 4-14：IPD 初始位置检测配置

⑥

参数名称	参数描述	数值	数值类型	量纲
ALIGN_ANGLE_M0	预定位角度	0.00	float	度
U_START_CUR_SET_F_M0	第一段定位电流	0.50	float	1A
U_START_CUR_SET_S_M0	第二段定位电流	2.00	float	1A
DC_HOLDTIME_TIME_LENTH_M0	第一段定位时间	500	s16	ms
DC_HOLDTIME_TIME_LENTH_STAGE1_M0	第二段定位时间	800	s16	ms
ALIGN_CURRENT_ACC_M0	定位电流加速调整值	0.50	float	A/s
ALIGN_CURRENT_DEC_M0	定位电流减速调整值	0.50	float	A/s

用户可以设定预定位角度，一般设置为0

采用2段定位电流方式
大小和时间由电机和负载决定

图 4-15: 预定位参数配置

参数名称	参数描述	数值	数值类型	量纲
OPEN_ANGLE_TAG_FREQ_M0	开环拖动最终频率	15.00	float	1Hz
CLOSE2OPEN_FREQ_M0	开环运行频率	10.00	float	1Hz
MIN_RUN_FREQ_M0	最低运行频率	30.00	float	1Hz
CURRENTLOOP_CLOSE_FREQ_M0	电流闭环频率	0.00	float	1Hz
FREQ_ACC_M0	开环拖动频率加速调整值	10.00	float	1Hz/s
FREQ_DEC_M0	开环拖动频率减速调整值	10.00	float	1Hz/s
OPEN2CLOSE_RUN_COV_TIME_M0	开环闭环过渡时间	30	u16	1ms

根据负载特性决定

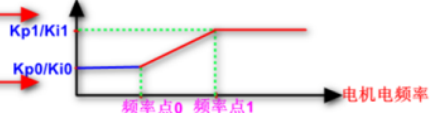
切闭环速度满足条件
当速度下降到这个值的时候进入开环
对给定的速度值有一个最小的限制

在SLVC过程中，电机速度小于这个值
时，不适用电流环，使用VF控制，当速度
大于这个值使用电流环控制

参数名称	参数描述	数值	数值类型	量纲
ID_START_M0	D轴启动电流设定	1.00	float	1A
IQ_START_M0	Q轴启动电流设定	1.00	float	1A
IQ_BRAKE_SET_M0	Q轴制动电流设定	0.00	float	1A
U_UNDVOLT_START_M0	母线电压保护电压值	0.00	float	1V
U_UNDVOLT_STOP_M0	母线电压保护结束值	0.00	float	1V
U_UNDVOLT_RECOVER_M0	母线电压保护恢复值	0.00	float	1V
U_UNDVOLT_PROT_ENABLE_M0	电压保护模式开启	0	u8	bool

参数名称	参数描述	数值	数值类型	量纲
THETA_FIL_TIME_M0	角度滤波时间	2.00	float	1ms
SPEED_FIL_TIME_M0	速度滤波时间	2.00	float	1ms
PLL_KP_GAIN_M0	PLL_Kp 估算器Kp0	16	s16	
PLL_KI_GAIN_M0	PLL_Kp 估算器Ki0	4	s16	
PLL_K_FREQ0_M0	PLL频率点0	50.0	s16	1Hz
PLL_KP_RUN_M0	PLL_Kp 估算器Kp1	20	s16	
PLL_KI_RUN_M0	PLL_Kp 估算器Ki1	4	s16	
PLL_K_FREQ1_M0	PLL频率点1	70.0	s16	1Hz

滤波时间越大，数据越平滑，
但会导致滞后，由实际调试效果修改此值



在频率0以下，使用Kp0/Ki0，在频率1以上使用Kp1/Ki1，中间使用二者的线性叠加

图 4-16: SLVC 强拖参数配置

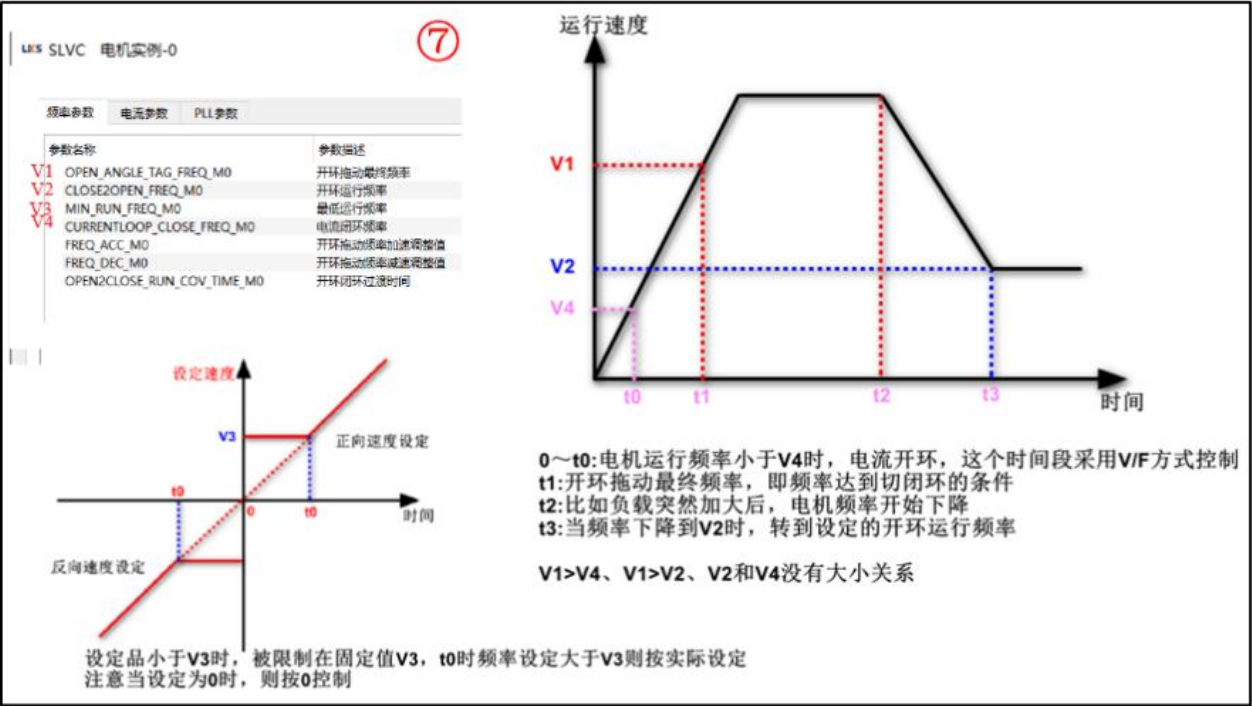


图 4-17: SLVC 频率参数说明

⑧

VF的功能已经在⑦中SLVC中实现, 这里是灰色的

参数名称	参数描述	数值	数值类型	量纲
VF_START_VALUE_M0	VF启动电压值	1.00	s16	V
VF_OPEN2CLOSE_TIME_M0	启动开环延时时间	1	u16	ms
VF_VOLT_ACC_M0	VF电压加速调整值	1.00	s16	V/ms
VF_VOLT_DEC_M0	VF电压减速调整值	1.00	s16	V/ms

图 4-18: VF 启动

⑨

电流环

速度环

功率环

控制模式选择

电流环Q轴电流设定值

实际输出电压0.8*母
线电压采样最大值

参数名称	参数描述	数值	数值类型	量纲
IQ_SET_M0	Q轴电流设定值	0.500	float.3	1A
VQMAX_M0	Q轴电压最大值	0.80	float.2	1
VQMIN_M0	Q轴电压最小值	-0.80	float.2	1
VDMAX_M0	D轴电压最大值	0.80	float.2	1
VDMIN_M0	D轴电压最小值	-0.80	float.2	1
P_CURRENT_KP_M0	Kp	1500	integer	1
P_CURRENT_KI_M0	Ki	780	integer	1
AUTO_FW_LIM_M0	最大弱磁电流	0.00	float.2	1A
D_CURRENT_REF_FIL_TIME_M0	D轴电流指令滤波时间	2.0	float.1	1ms
Q_CURRENT_REF_FIL_TIME_M0	Q轴电流指令滤波时间	2.0	float.1	1ms

图 4-19: 环路-电流环参数配置



图 4-20：环路-速度环参数配置

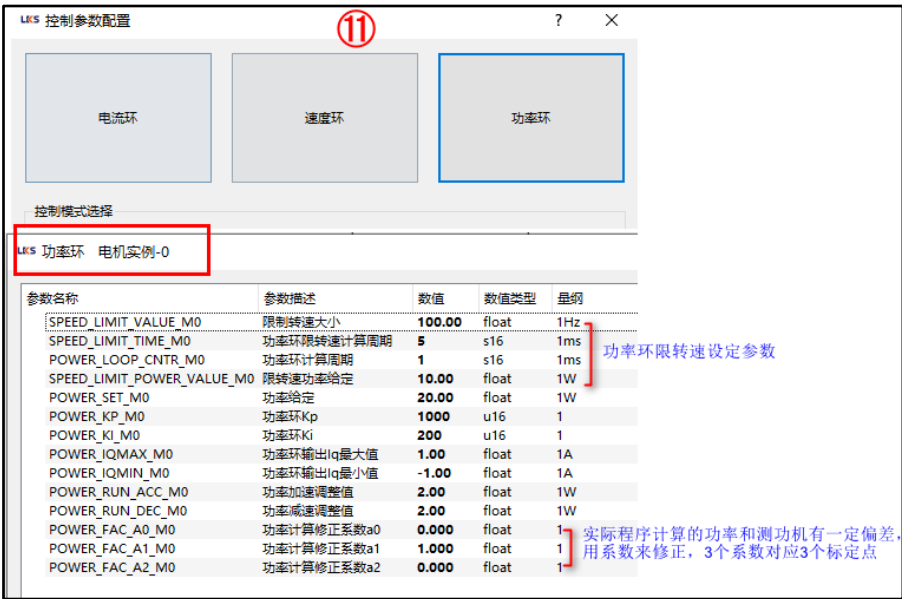


图 4-21：环路-功率环参数配置



图 4-22：温度检测

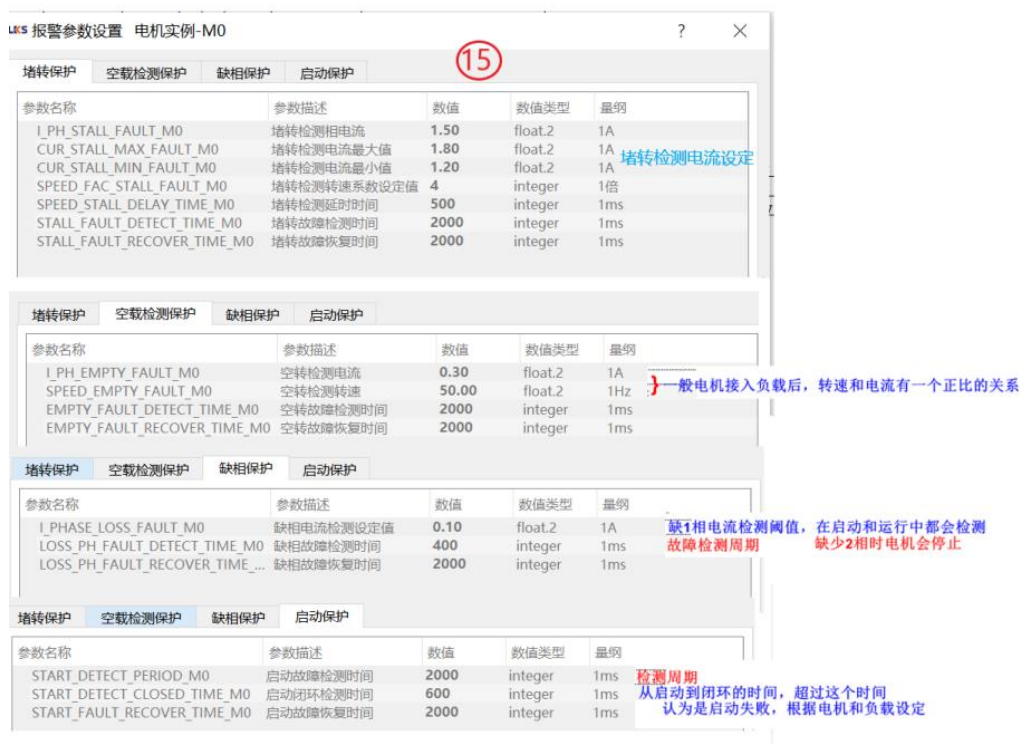


图 4-23：报警参数配置

<5>lksmotor 参数烧录

在配置完参数后下载配置参数到 MCU 中，下载方法如图 3-30，注意：这些参数是写到 MCU 的 flash 中，占用 flash 空间的最后 512 Bytes，所以要确保工程代码生成文件+配置参数 总大小不超过 MCU 的 flash 大小；

按照图 3-31 中①-②-③-④的步骤下载配置参数到 MCU 中，最后进度条提示 100%下载完成，对于下载过程中的提示信息做简单说明；

下载参数时，首先程序会自动判断电机是否处于停止状态（有一个检测判断阈值），如果判断电机处在转动状态，则等待直至检测停止值满足要求，如果电机惯量较大，则等待时间较长；

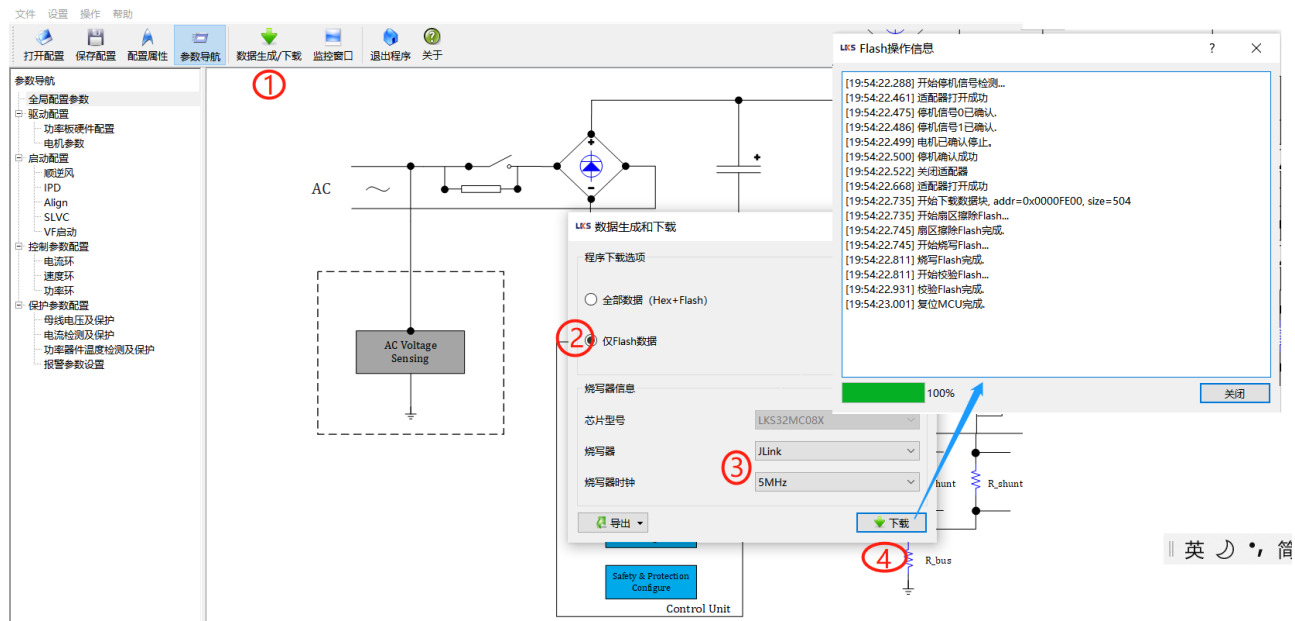


图 4-24: lksmotor 参数下载步骤

5.一般调试步骤总结

新平台代码是凌鸥官方针对电机控制推出的针对不同应用下的电机的驱动 sdk 程序，方便对于不太了解电机控制原理的用户上升进行开发调试电机。

用户利用新平台进行开发时，可以先按照如下调试步骤，熟悉调试流程，文档对于每一步骤的操作和结果作了详细的描述。

表 5.1 调试步骤

调试步骤总结	
步骤 1: 驱动通路测试	说明: 打开驱动测试函数, 验证驱动通路
步骤 2: 保护参数验证	说明: 母线电压分压比, 硬件过流配置与验证
步骤 3: VF 采样通路测试	说明: 匹配电机参数, 采样电阻, 放大倍数验证采样
步骤 4: IF 电流环闭环运行	说明: 进一步验证采样以及电流环闭环效果
步骤 5: 启动调试	说明: 调节启动参数配置, 保证电机稳定启动
步骤 6: 速度环调试	说明: 速度环 KP, KI 参数
步骤 7: 恒功率调试	说明: 采样母线电流计算功率
步骤 8: 增加串口等逻辑	说明: 用户其他逻辑功能添加

5.1 mcpwm 输出通路测试与硬件过流保护配置

首先根据自己的硬件型号（MCU + 功率板），选择对应的代码模板，03 系列和 08 系列可以选择不同编译器版本的代码，45 系列选择 ac5 编译器版本的代码，如果是对应代码匹配的官方的 mcu+功率驱动板，那么不需要修改相关硬件配置，可以直接进入下载运行、调试电机，如果硬件有所更改，需要进行硬件参数匹配，具体按照以下步骤进行匹配检查

名称	修改日期	类型
1、LKS32MC03x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
1、LKS32MC07x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
1、LKS32MC08x_1Motor1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹
LKS32MC45x_2Motor2R_1R_code+Clib_R_V02_0_9a_AC5	2023/11/24 11:54	文件夹

图 5.1 选择对应的模板例程

1.mcu 型号宏定义更改，因为不同型号的 mcu 存在是否内置预取等区别，需要在模板工程中将对应的 mcu 宏定义改成自己型号的 mcu，如果版本更新招不到对应型号，可以根据芯片是否内置预驱，选择对应的宏定义极性，对应的位置在 hardware_config.h 文件中

```
#define CHIP_PART_NUMBER LKS32MC081 /* 芯片型号选择 */
```

图 5.2 修改对应 mcu 型号宏定义

2.fail 保护事件配置：mcpwm 具有硬件保护功能，当发生 fail 事件的时候，对应的 mcpwm 会自动关断输出，fail 事件的来源可以来源于外部 IO 或者比较器的信号，默认的程序模板中根据 demo 板配置为使用 FAIL0 保护,且来源于比较器信号，在读取 FAIL 事件标志时，只读取 FAIL0 对应的 BIT4,实际硬件如果 fail 保护事件有所更改，要根据实际 fail 事件，进行对应比较器和 fail 位的设置，同时要将 pwm 初始化中的对应 fail 事件的输入使能，配置的函数分别是 hardware_init.c 的 CMP_init()和 MCPWM_init(), 对应的获取 fail 标志清 fail 标志函数为 HAL_DEF.c 的 GetEPWM0_breakInStatus 和 ClrEPWM0_breakInStatus

```
void CMP_init(void)
{
    CMP_InitTypeDef CMP_InitStruct;

    CMP_StructInit(&CMP_InitStruct);

    CMP_InitStruct.CMP0_EN      = ENABLE;          /* 比较器 0 开关 */
    CMP_InitStruct.CMP0_SELN    = SELN_DAC;
    CMP_InitStruct.CMP0_SELP    = SELP_CMP_OPA0IP; /* CMP0_P CMP0_OPA0IP */
    CMP_InitStruct.CMP0_IN_EN   = DISABLE; /* 比较器信号输入使能 */
    CMP_InitStruct.CMP0_IE      = DISABLE;          /* 比较器 0 信号中断使能 */

    CMP_InitStruct.CMP1_EN      = DISABLE;          /* 比较器 1 开关 */
    CMP_InitStruct.CMP1_SELN    = SELN_DAC;          /* CMP1_N 内部参考 */
    CMP_InitStruct.CMP1_SELP    = SELP_CMP_IP0; /* CMP1_P CMP1_OPA3 */
}
```



```

    CMP_InitStruct.CMP1_IN_EN    = DISABLE;    /* 比较器信号输入使能 */
    CMP_InitStruct.CMP1_IE      = DISABLE;      /* 比较器 1 信号中断使能 */

    CMP_InitStruct.FIL_CLK10_DIV16 = 15;        /* 即滤波宽度=tclk 周期*16*CMP_FltCnt */
    CMP_InitStruct.CLK10_EN = DISABLE;          /* 时钟使能 */

    CMP_Init(&CMP_InitStruct);
}

```

图 5.3 比较器配置（根据实际用的比较器配置）

```

/*****
函数名称:    void GetEPWM0_breakInStatus(STR_PubDatHDMD* tS_pHd2Md)
功能描述:    读取 breakin 信号
输入参数:    tS_pHd2Md
输出参数:    无
返 回 值:    无
其它说明:    查看是否存在过流事件
修改日期    版本号        修改人        修改内容
-----
2024/5/27    V1.1            TongHua Li    整理
*****/
void GetEPWM0_breakInStatus(STR_PubDatHDMD* tS_pHd2Md)
{
    tS_pHd2Md->bBreakInStatus = (MCPWM{EIF} & BIT4);
}

/*****
函数名称:    void ClrEPWM0_breakInStatus(void)
功能描述:    清除 breakin 信号
输入参数:    无
输出参数:    无
返 回 值:    无
其它说明:
修改日期    版本号        修改人        修改内容
-----
2024/5/27    V1.1            TongHua Li    整理
*****/
void ClrEPWM0_breakInStatus(void)
{
    MCPWM{EIF} = (MCPWM{EIF} | BIT4);
}

/**fail 事件配置，位置 hardware_init.c 的 MCPWM_init()
MCPWM_InitStructure.FAIL0_INT_EN = DISABLE;          /* FAIL0 事件 中断使能或关闭 */
MCPWM_InitStructure.FAIL0_INPUT_EN = ENABLE;          /* FAIL0 通道急停功能打开或关闭 */
MCPWM_InitStructure.FAIL0_Signal_Sel = FAIL_SEL_CMP; /* FAIL0 事件信号选择，比较器或 IO 口 */
MCPWM_InitStructure.FAIL0_Polarity = HIGH_LEVEL_ACTIVE; /* FAIL0 事件极性选择，高有效 */

MCPWM_InitStructure.FAIL1_INT_EN = DISABLE;          /* FAIL1 事件 中断使能或关闭 */
MCPWM_InitStructure.FAIL1_INPUT_EN = DISABLE;        /* FAIL1 通道急停功能打开或关闭 */
MCPWM_InitStructure.FAIL1_Signal_Sel = FAIL_SEL_CMP; /* FAIL1 事件信号选择，比较器或 IO 口 */
MCPWM_InitStructure.FAIL1_Polarity = HIGH_LEVEL_ACTIVE; /* FAIL1 事件极性选择，高有效或低有效 */

```

图 5.4 fail 事件配置

注:MCU 的 MCPWM 输出极性，是否电平反转、是否交换 PN 等在用户选定 mcu 型号以后，一般会自动将对应的宏打开，如果硬件开环调试时，不能正确输出 pwm，需要检查相关 mcpwm 配置

3.DAC 硬件限流保护设置：用户需要根据实际的运行电流，设置对应的硬件限流值，这里的宏定义的硬件过流保护的电压值单位是 mv，对应的宏定义在 MC_Parameter_M0.h 的 I_PH_HARD_OC_FAULT_CMP_VOLT_M0

```
/* 过流检测参数 */
#define I_PH_HARD_OC_FAULT_CMP_VOLT_M0      (400)          /* 单位：mV 硬件过流保护比较电压值 */
#define I_PH_OVERCURRENT_FAULT_M0           (3.5)          /* 单位：A 软件过流检测设定值 */
#define I_PH_OVERCURRENT_FAULT_TIMES_M0     (5)            /* 单位：软件过流检测次数 */

#define CURRENT_FAULT_RECOVER_TIME_M0       (1000)         /* 单位：ms 过流恢复时间 */
```

图 5.5 硬件限流值设定

4.MCPWM 硬件通路测试:用户可以通过调用 DebugPWM_OutputFunction();或者将 DEBU G_PWM_OUTPUT 宏打开，来测试硬件 MCPWM 通路，将 U, V, W 设置成不同的占空比，比如 50%，25%，12.5%。通过示波器，测量 u, v，三相正常输出对应的占空比，验证 MCPWM 通路。

```
void DebugPWM_OutputFunction(void)
{
    MCPWM_TH00 = (s16) (- (PWM_PERIOD_M0 >> 2));
    MCPWM_TH01 = (PWM_PERIOD_M0 >> 2);
    MCPWM_TH10 = (s16) (- (PWM_PERIOD_M0 >> 2));
    MCPWM_TH11 = (PWM_PERIOD_M0 >> 2);
    MCPWM_TH20 = (s16) (- (PWM_PERIOD_M0 >> 2));
    MCPWM_TH21 = (PWM_PERIOD_M0 >> 2);

    PWMOutputs (ENABLE);
    while(1)
    {
    }
}
```

图 5.6 Debug_PWM 函数

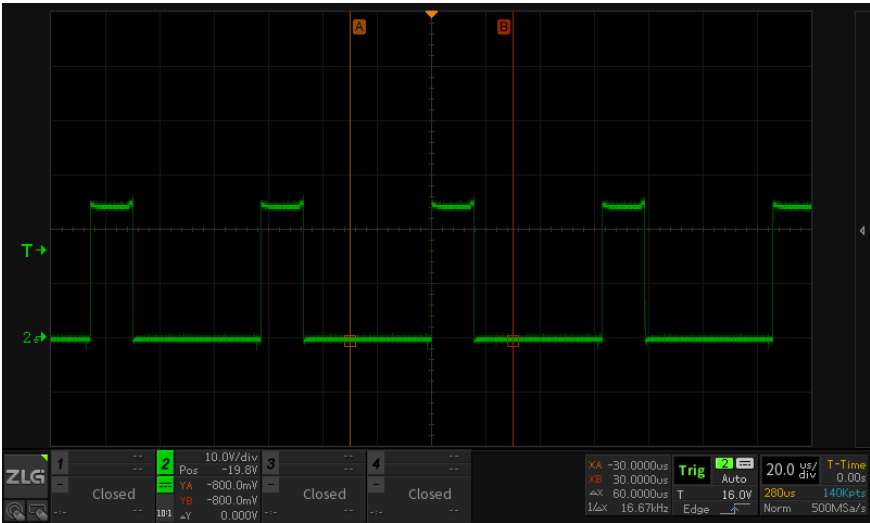


图 5.7 mcpwm 通路验证

注：不同的硬件，采用 mcpwm 通道控制对应的控制 u, v, w 顺序可能不同，比如程序中配

置 mcpwm 的通道 0 输出 50% 的占空比，通道 1 输出 25% 的占空比，实际示波器在 V 相测量到 50% 的占空比，U 相测量到 25% 的占空比，可以判断 mcpwm 的通道 0 控制的板子的 V 相，通道 1 控制的板子的 U 相，如果是三或双电阻采样，则需要调整下面对应的 MCPWM 寄存器更新函数，使得通道和对应的相电流一致，修改位置在 HAL_Drv.h 的 HAL_DRV0_MCPWM_RegUpdate 函数

```
static inline void HAL_DRV0_MCPWM_RegUpdate(STR_PubDatMDHD* tS_pMdToHd)
{
    MCPWM_TH00 = tS_pMdToHd->mMcPwm.nuTimePhBN;
    MCPWM_TH01 = tS_pMdToHd->mMcPwm.nuTimePhB;

    MCPWM_TH10 = tS_pMdToHd->mMcPwm.nuTimePhAN;
    MCPWM_TH11 = tS_pMdToHd->mMcPwm.nuTimePhA;

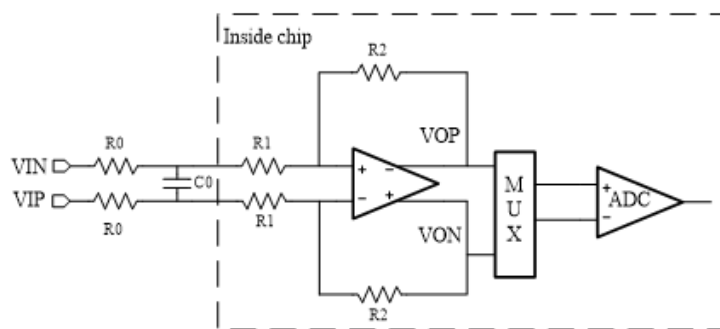
    MCPWM_TH20 = tS_pMdToHd->mMcPwm.nuTimePhCN;
    MCPWM_TH21 = tS_pMdToHd->mMcPwm.nuTimePhC;

    MCPWM_TMR0 = tS_pMdToHd->mMcTmr.nuPoint0; // 第一个采样点
    MCPWM_TMR1 = tS_pMdToHd->mMcTmr.nuPoint1; // 第二个采样点
}
```

图 5.8 mcpwm 通道匹配

5.2 开环测试:

开环测试之前，要保证软件的放大倍数与采样电阻配置与硬件电路一致，放大倍数由下图所示的内部运放的反馈电阻值和外部输入电阻决定，其中 R2/R1 的值用户可以更改 OPA_SELECT_M0 宏定义进行选择，外部的 R0 由外部硬件决定，放大倍数的计算公式 $Gain = R2 / (R1 + R0)$ ，用户选择合适的挡位以后，配置对应的放大倍数宏定义在 MC_Parameter_M0.h 的 AMPLIFICATION_GAIN_M0



```
#define AMPLIFICATION_GAIN_M0 (17.5439) /*运放放大倍数*/
```

图 5.9 运放配置

采样电阻的大小与电流标么的基准值相关，采样电阻需要根据硬件进行匹配如下图，位置在

MC_Parameter_M0.h 的 RSHUNT_M0

```
#define RSHUNT_M0 (0.05) /* 单位: Ω 采样电阻阻值 */
```

图 5.10 采样电阻配置

ADC 采样通道与开环测试,adc 采样通路一般不更改该通道采样顺序(相电流/母线电流),修改电机相电流采样顺序和运放的关系,根据实际的双电阻和单电阻采样更改对应的通道,修的位置在 hardware_config.h 中

```
#define ADC0_CURRETN_A_CHANNEL (ADC0_CHANNEL_OPA1) //2
#define ADC0_CURRETN_B_CHANNEL (ADC0_CHANNEL_OPA2) //1
#define ADC0_CURRETN_C_CHANNEL (ADC0_CHANNEL_OPA0)

#define ADC_1SHUNT_CURR_CH (ADC0_CHANNEL_OPA3) /* 单电阻采样电流通道 */

#define ADC_BUS_VOL_CHANNEL (ADC_CHANNEL_12) /* 母线电压 ADC 采样通道 */
#define M0_ADC_BUS_CURR_CH (ADC0_CHANNEL_OPA3) /* 母线电流 ADC 采样通道 */

#define ADC0_TEMP_CHANNEL (ADC_CHANNEL_7) /* 温度检测 */
```

图 5.11 adc 采样通道配置

开环运行模式设置,在验证 adc 采样通路的时候,可以将电机设定在 VF 模式下,具体的设置方式为为以下步骤:

- 将环路设定为速度环模式,修改位置在 MC_GlobalDef_M0.h 的 CLOSE_LOOP_M0

```
#define CLOSE_LOOP_M0 (SPEED_LOOP) // 环路选择
```

图 5.12 环路设置

- 开环拖动最终频率>电流环闭环频率>速度环设定频率,这里设定为 15hz,设定方式可以利用 lksmotor 进行设置,也可以在程序中修改,修改的位置在 MC_Parameter_M0.h 的 CURRENTLOOP_CLOSE_FREQ_M0

```
#define CURRENTLOOP_CLOSE_FREQ_M0 (0.0) /* 单位: Hz 电流环闭环频率 */
```

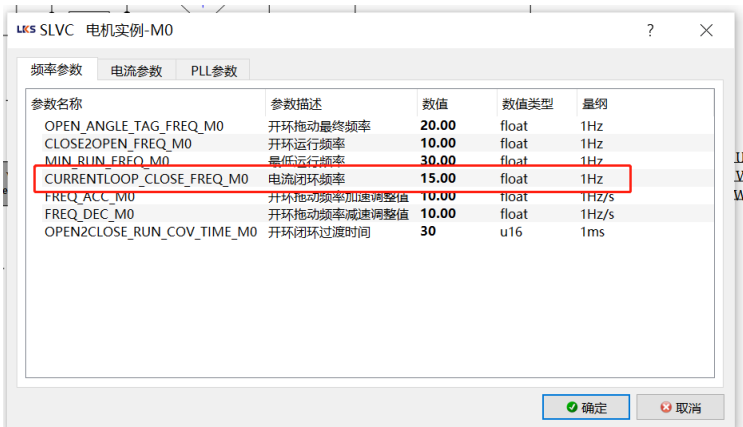


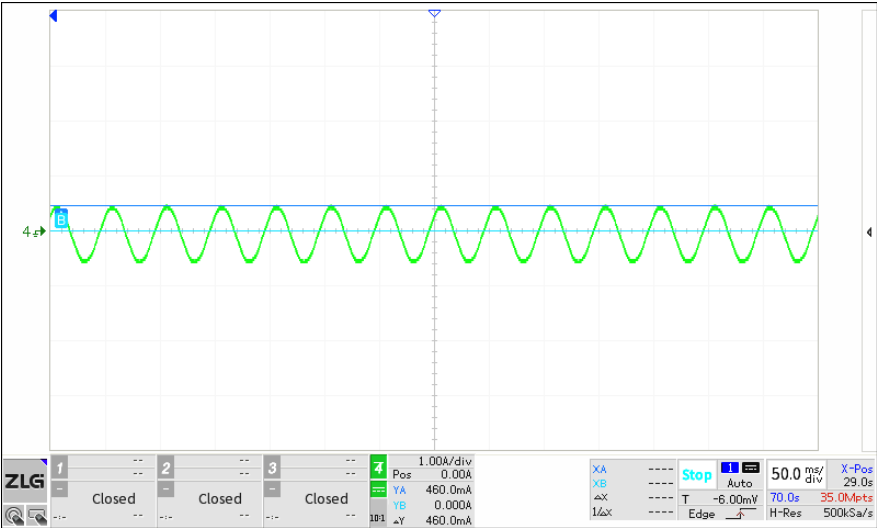
图 5.13 电流闭环频率两种设定方式

- 配置 RTT 读取电流采样值,观测 VF 运行波形,通过示波器查看 VF 相电流幅值的波形是

460mA,通过 RTT 查看采样的相电流峰值图 5.14 为 1666, 通过转换 $1666/32767*9.9= 0.503\text{A}$, 其中 9.9 为最大采样电流 I_{max} , I_{max} 的计算方式为如下

$$I_{\text{max}}=(\text{AD 量程}/(\text{放大倍数}*\text{采样电阻}))$$

开环拖动的目的是验证验证采样通路和放大倍数的正确性, 同时用户需要注意, RTT 配置的显示 U, V 两相的电流, 所以红色曲线应该超前绿色曲线 120 度, 如果不一致需要检查是不是采样通道顺序配置有问题。



```
#if(RTT_FUNCTION == FUNCTION_ON)
Rttstru.data0 = getElectSpdEst(0);
Rttstru.data1 = getElectAngleEst(0);
Rttstru.data2 = getPhaseCur_U(0);
Rttstru.data3 = getPhaseCur_V(0);

SEGGER_RTT_Write(1, &Rttstru, 8);
#endif
```

图 5.14 配置 RTT 查看相电流

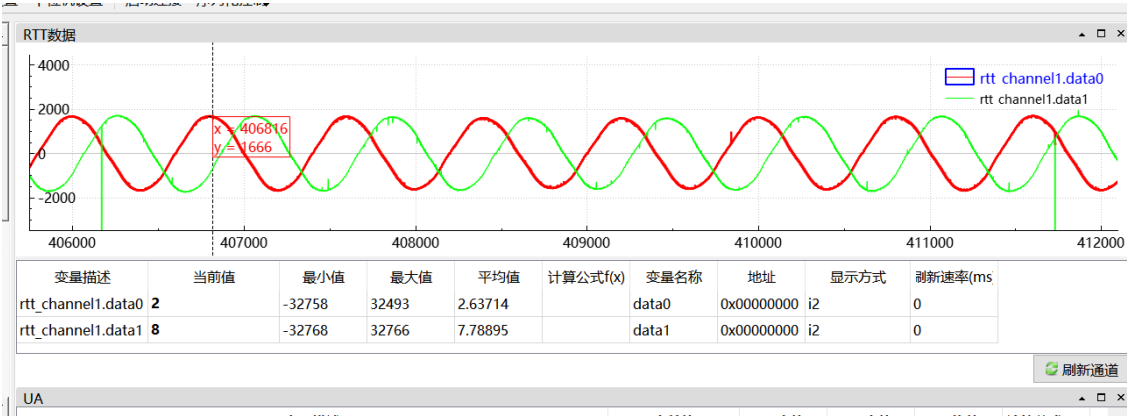


图 5.15 开环 RTT 波形

注: VF 功能的说明

1. 当给定的频率值 $f <$ 电流闭环频率值时, 实现 VF 的拖动功能, 电流闭环不工作。

- 当给定频率值 $f \geq$ 电流闭环 时，电流闭环开始工作，VF 功能是实现解耦补偿。当不需要进行解耦补偿时，可以通过代码或者宏编译去除 VF 功能。

注：本步骤的目的是验证电流采样通路没有问题，有的电机可能拖起来不是完美的正弦，但是要确保 RTT 抓取的数值转为实际的相电流值和示波器上测量的信号幅值、趋势是一样的

5.3 电流强拖验证：

该步骤是电机运行在电流环闭环，角度开环的模式，可以指定拖动电流大小，拖动频率，具体配置步骤如下：

- 环路设置在速度环模式
- 开环拖动最终频率 > 速度环设定频率 > 电流环闭环频率
- 设置拖动电流的大小，对应程序中的 ID_START_M0 和 IQ_START_M0，用户可以根据实际调试情况，id，iq 给拖动电流值，电机在该模式下运行，实际的相电流峰值应该为

$$\sqrt{id^2 + iq^2}$$

例如下面的配置开环拖动频率为 30hz，电流环闭环频率为 0hz，拖动电流为 0.5A（修改位置在 MC_Parameter_M0.h 的 ID_START_M0 和 IQ_START_M0），用户可以给速度给定频率值为 20hz，那么电机将按照 0.5A 的峰值电流，20hz 拖动运行

```
#define CURRENTLOOP_CLOSE_FREQ_M0          (0.0)          /* 单位：Hz 电流环闭环频率 */

/*-----开环闭环切换过渡参数-----*/
#define OPEN2CLOSE_RUN_COV_TIME_M0         (30)           /* 开环闭环切换过渡时间：单位：mS 30*/
#define OPEN2CLOSE_RUN_CURRENT_RAMP_M0     (0.05)         /* 开环闭环切换过渡内，D,Q 轴电流变化斜率。
单位：A/ms */

#define ID_START_M0                         (1.0)          /* 单位：A， D 轴电流设定值 */
#define IQ_START_M0                         (1.0)          /* 单位：A， Q 轴电流设定值 */
```

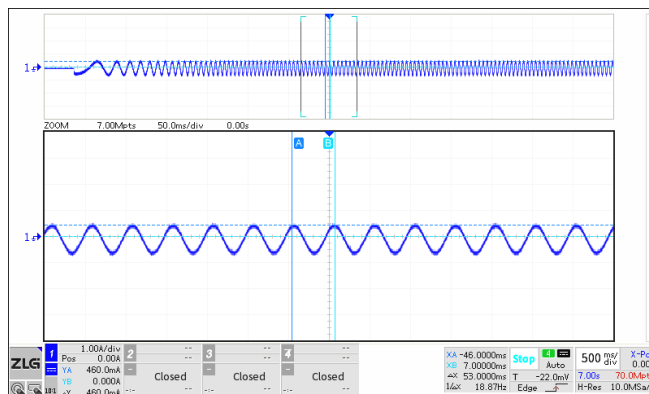


图 5.16 实际相电流波形



5.15 电流强拖模式下的 RTT 相电流波形

5.3 电流环调试

用户更改环路宏定义见图 5.8，然后可以在 lksmotor 或者程序中的宏定义修改电流环路的 iq 设定值，修改位置在 MC_Parameter_M0.h 的 IQ_SET_M0，也可以将电流宏定义设定值自定义为变量在上位机中实时修改，观测电流的输出跟随情况。

```
#define IQ_SET_M0 (1.0) /* 单位: A IqRef, Iq 给定值 */

#define VQMAX_M0 (0.8) /* Q 轴最大输出限制 */
#define VQMIN_M0 (-0.8) /* Q 轴最小输出限制 */

#define VDMAX_M0 (0.8) /* D 轴最大输出限制 */
#define VDMIN_M0 (-0.8) /* D 轴最小输出限制 */

#define P_CURRENT_KP_M0 (0) /* 电流环 Kp, 实际运用的 Kp 会根据这个值和电机参数计算出最终的 Kp */
/* 0 --- 自动计算参数 非 0 --- 设定值 */

#define P_CURRENT_KI_M0 (1500) /* 电流环 Ki, 实际运用的 Kp 会根据这个值和电机参数计算出最终的 Ki */
```

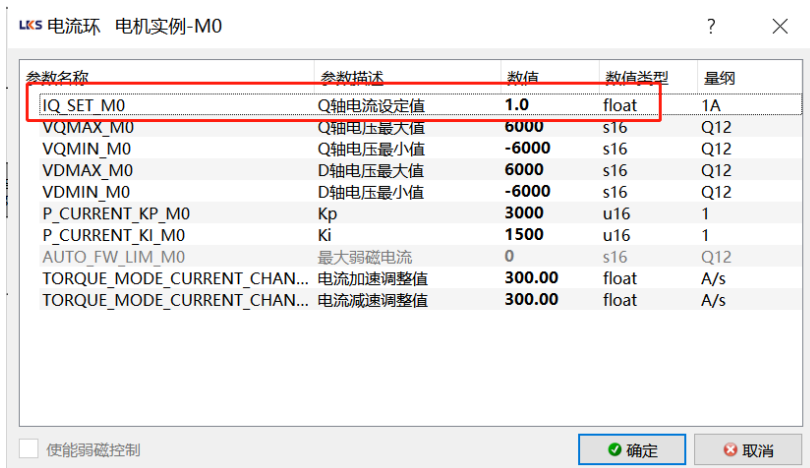



图 5.17 更改电流环 iq 设定值

5.4 速度环调试

默认模板中，已经将速度频率给定值定义为可修改变量，用户可以在上位机中修改速度频率值，观测速度反馈的跟随情况

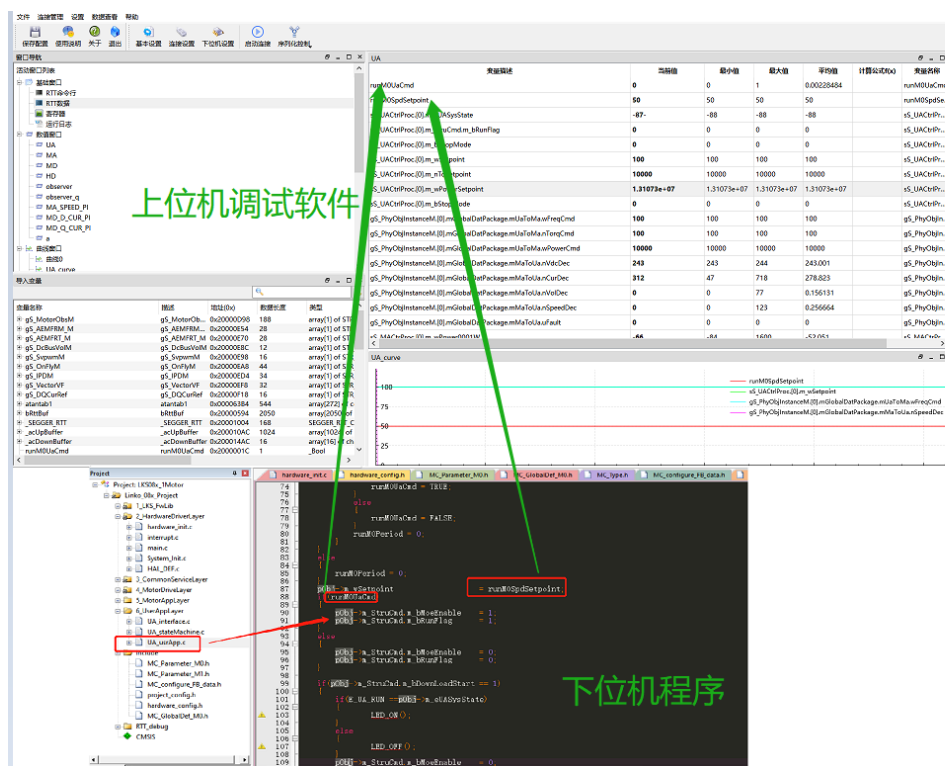


图 5-18: 运行速度设定

速度环 KP, KI 的计算用户可在文件中使能自动计算功能如下，也可在参数配置中设定，修改位置在 Main_Drive.c 的 StateInit 函数中

```
#if 1
t_nTmp = pDrvCfgPara->mS_FBSlvcCfg0.m_wSpeedFilTime + 25;
//自动调节速度环 PI 功能
SpeedPIAutoTunning(pObj->mg_nMACtrlProcID,
                    9000,
```

```

t_nTmp,
&t_wTmpKp,
&t_wTmpKi);

pObj->m_pMotorSpeed->mSpeedPI.KP = sat(t_wTmpKp,0,32767); // PI 参数限幅
pObj->m_pMotorSpeed->mSpeedPI.KI = sat(t_wTmpKi,0,32767);
#endif

```

图 5.19 速度环 PI 参数自动计算使能

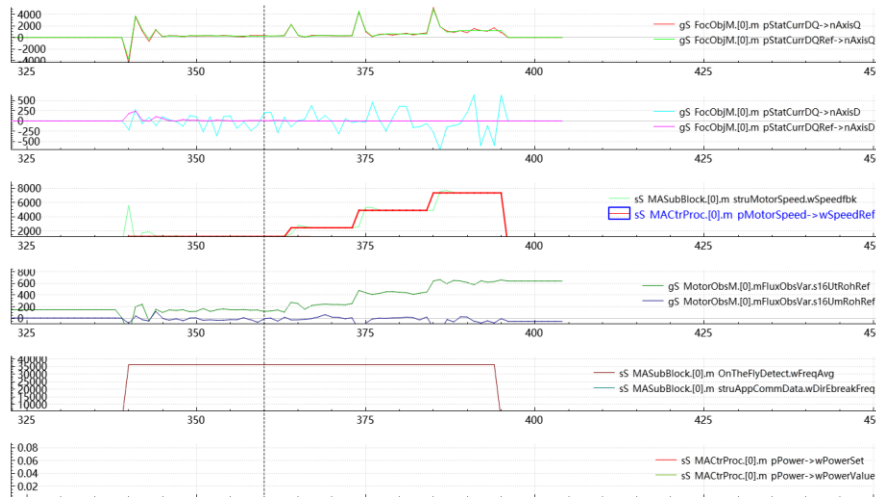


图 5.20 速度环调速跟随曲线

注：快速闭环模式说明

降低闭环速度设定值、设置较快的开环加速度，设定 **Id** 电流启动值，可以实现不对相、不打初始位置情况下快速的闭环启动，在不使能顺逆风功能下可以直接顺风启动、逆风启动。

具体在风扇类应用中可以将闭环频率设定值设为 **0**，设置较快的加速度，设定合适的 **Id** 电流启动值，具体设置的地方如下图所示，首先根据电机的额定频率设置合适的开环拖动频率，然后设置较快的加速度，测试合适的 **id** 电流启动值，可以实现快速闭环启动。电机启动参数修改的位置在 **MC_Parameter_M0.h**

```

/*-----开环参数-----*/
#define OPEN_ANGLE_TAG_FREQ_M0 (15.0) /* 单位: Hz 开环拖动最终频率 */
#define FREQ_ACC_M0 (20.0) /* 单位: Hz/s 开环拖动频率加速调整值 */
#define FREQ_DEC_M0 (20.0) /* 单位: Hz/s 开环拖动频率减速调整值 */

#define MATCH_TIME_M0 (5) /* 估算和给定电流匹配次数，当前未用，预留 */

#define MIN_RUN_FREQ_M0 (30.0) /* 单位: Hz 最低速度 */
#define CLOSE2OPEN_FREQ_M0 (2.0) /* 单位: Hz 切换为开环拖动频率 */
#define CURRENTLOOP_CLOSE_FREQ_M0 (0.0) /* 单位: Hz 电流闭环频率 */

/*-----开环闭环切换过渡参数-----*/
#define OPEN2CLOSE_RUN_COV_TIME_M0 (30) /* 开环闭环切换过渡时间: 单位: ms 30 */
#define OPEN2CLOSE_RUN_CURRENT_RAMP_M0 (0.05) /* 开环闭环切换过渡内, D, Q 轴电流变化斜率。

```

```
单位: A/ms */

/* 当前未用, 预留 */

#define ID_START_M0 (1.0) /* 单位:A, D 轴电流设定值 */
#define IQ_START_M0 (1.0) /* 单位:A, Q 轴电流设定值 */
```

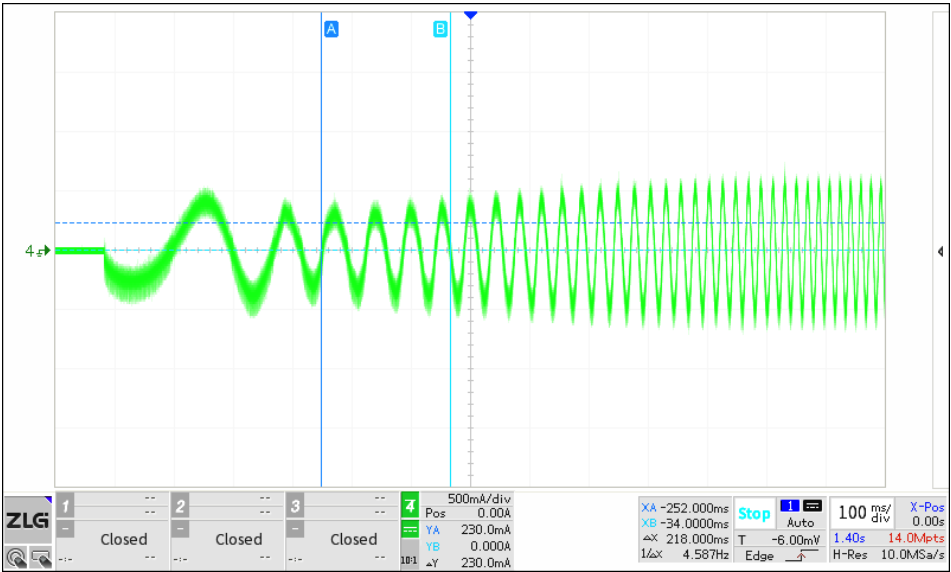


图 5.21 快速闭环启动

5.5 功率环调试

用户选择环路定义为 POWER_LOOP,则电机运行在功率环模式下, 修改位置在 MC_GlobalDef_M0.h 的 CLOSE_LOOP_M0

```
#define CLOSE_LOOP_M0 (POWER_LOOP) // 环路选择
```

图 5.22 功率环修改

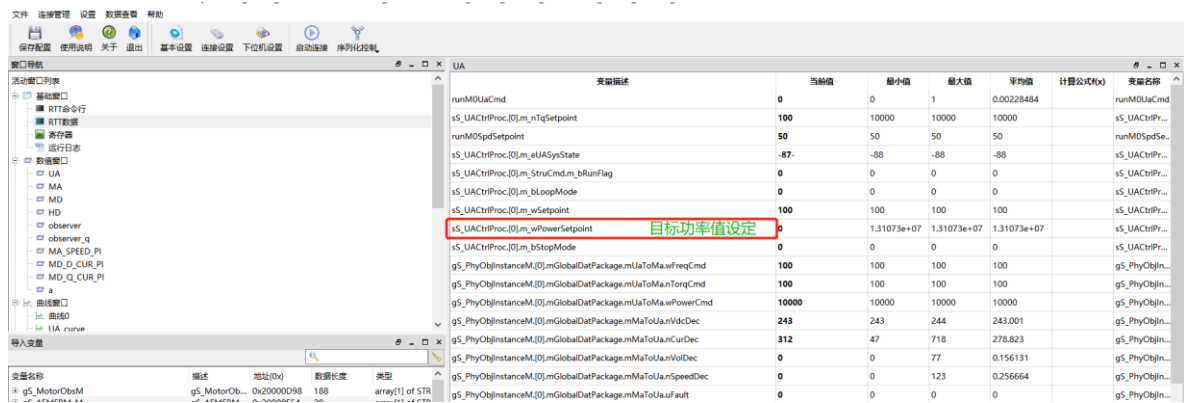


图 5-23：功率值设定

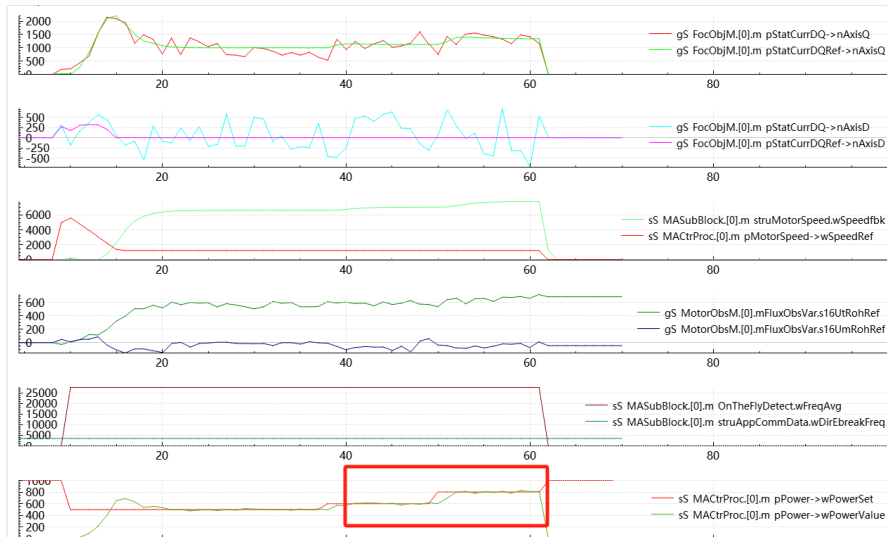


图 5.24 功率环跟随曲线

5.6 故障保护功能与恢复

电机控制中，根据不同运用场景，会遇到一些常见故障，如软硬件过流、过欠压、堵转等（见 STR_FaultStatus 结构体，如图 5.24 所示），对应的位置在 fault_detection.h。新平台程序中对这些故障进行了判断和处理，发生故障时，会使相应故障标志置位。

```
typedef union
{
    u16 R;
    struct
    {
        u16 SoftCurretError : 1; //软件过流标志 0x0001
        u16 HardCurretError : 1; //硬件过流标志 0x0002
        u16 VoltOverError : 1; //过压故障标志 0x0004
        u16 VoltUnderError : 1; //欠压故障标志 0x0008
        u16 StallError : 1; //堵转故障标志 0x0010
        u16 EmptyError : 1; //空转故障标志 0x0020
        u16 PhaseLossError : 1; //缺相故障标志 0x0040
        u16 TempOverError : 1; //温度故障标志 0x0080
        u16 OffsetError : 1; //偏置故障标志 0x0100
        u16 StartError : 1; //启动故障标志 0x0200
        u16 : 6; //预留
    } B;
} STR_FaultStatus;
```

图 5.25 故障码

在 MC_Parameter_M0.h 头文件内可进行这些故障参数的设置。下面列出几种常见的故障判断条件：

➤ 硬件过流：

目前 Demo 版上硬件过流主要方式：使用比较器(CMP)产生过流信号，触发 MCPWM 的 Fail 事件。同时 Fail 事件也可以通过外部引脚触发，如使用 IPM 传来的过流信号。发生 Fail 事件后，MCPWM 进入默认输出模式，即关断。硬件过流的配置可以见 5.1 章节。

➤ 软件过流

软件过流保护主要是通过读取三相相电流，如果三相电流瞬时值大于设定阈值，累加一定时间后判断为软件过流。对应的位置在 fault_detection.c 的 FaultCurrentCheck 函数在 MC_Parameter_M0.h 头文件内进行参数设置，代码如图所示，1ms 执行周期。

```
/* 过流检测参数 */
#define I_PH_HARD_OC_FAULT_CMP_VOLT_M0      (400)          /* 单位：mV 硬件过流保护比较电压值 */
#define I_PH_OVERCURRENT_FAULT_M0           (3.5)          /* 单位：A 软件过流检测设定值 */
#define I_PH_OVERCURRENT_FAULT_TIMES_M0     (5)            /* 单位：软件过流检测次数 */

#define CURRENT_FAULT_RECOVER_TIME_M0      (1000)          /* 单位：ms 过流恢复时间 */
```

图 5.26 软件过流参数设置

```
/* *****
 * 函数名   : FaultCurrentCheck(STR_VectorUVW *pstruCurrent, u8 t_bObj_ID)
 * 说明     : 软件过流检测
 * 设计思路 : 1. 如果三相电流绝对值超过设定的软件过流值，则判定为软件过流。
 * 参数     : STR_VectorUVW *pstruCurrent, u8 t_bObj_ID
 * 返回值   : 无
 * 修改时间 : 2022.11.22
 * 修改时间 : 2023.09.04
 *          在 AD 中调用时，会花费查找 MA 指针的时间
 *          如果对时限要求不严格，在 1ms 中调用执行 FaultCurrentCheckMA()
 * *****
void FaultCurrentCheck(STR_VectorUVW *pstruCurrent, u8 t_bObj_ID)
{
    PSTR_MACtrProc pObj = directGetMACtrPtr(t_bObj_ID);
    PSTR_DrvCfgPara pDrvCfgPara = pObj->mg_pCtrlObj->m_pDrvCfgPara;

    /*三相电流瞬时值大于设定阈值，累加一定时间后判断为软件过流*/
    if((ABS(pstruCurrent->nPhaseU) > pObj->m_pFaultValue->nOverCurrent)
        || (ABS(pstruCurrent->nPhaseV) > pObj->m_pFaultValue->nOverCurrent)
        || (ABS(pstruCurrent->nPhaseW) > pObj->m_pFaultValue->nOverCurrent)
    )
    {
        if(pObj->m_pstruFaultCurrentTime->nCheckCnt1++ >=
pDrvCfgPara->mS_FBCurSense.nSoftOvCurTimes)
        {
            pObj->m_pFaults->B.SoftCurretError = 1;          /*软件过流标志置 1*/
        }
    }
    else
    {
        pObj->m_pstruFaultCurrentTime->nCheckCnt1 = 0;
    }
}
```

图 5.27 软件过流检测

➤ 过欠压 FaultVoltageCheck(STR_MACtrProc *pObj)

当母线电压大于过压设定值，则判定为过压故障；当母线电压小于欠压设定值，则判定为欠压故障。滤波时间为 1s，用户可以在 MC_Parameter_M0.h 宏定义中修改。

➤ 离水空转 FaultEmptyCheck(STR_MACtrProc *pObj)

离水空转检测，在一般洗衣机、水泵类应用上使能使用，其他应用可以不使能，主要用来防止扇叶离水后，速度过快，磨损扇叶。

判断方式：如果 I_q 小于空转电流设定值，并且转速大于空转转速设定值，则判定为离水空转。

➤ 堵转检测 FaultStallCheck(STR_MACtrProc *pObj)

堵转保护主要有用于检测电机发生异常堵转时关机保护或者重启。判断逻辑如下：

1. 转速和电压比例异常，电压小，转速高，则判定为堵转；代码如图 5.28

```
wSpeedfromVolt = (getVolt2FreqCalcPara(pObj->mg_nMACtrlProcID) *
getMotorActVoltFromObjIdlms(pObj->mg_nMACtrlProcID)) >> 15;
wSpeedfromVolt1 = (wSpeedfromVolt * pFaultValue->wStallSpeedfactor) >> 8;

if( wSpeedfbkABS > wSpeedfromVolt1 ) //电机电压和转速不匹配，电压小转速高，则判定为堵转。
{
    if(pStallFaultTime->nCheckCnt2 < pDrvCfgPara->mS_FBEmptyStall.nStallDetectTime) //滤波时间 5ms
    {
        pStallFaultTime->nCheckCnt2 += FAULT_DETECT_CYCLE;
    }
    else //判定为电机堵转
    {
        pStallFaultTime->nCheckCnt2 = 0;
        pObj->m_pFaults->B.StallError = 1; //堵转错误标志位置 1
        pFaultValue->nStallFlag = 2; //堵转标志位为 2，用于区分触发了哪种堵转
    }
    else
    {
        pStallFaultTime->nCheckCnt2 = 0;
    }
}
```

图 5.28 堵转检测方式一

2. 电机电流大于堵转最大电流，判定为堵转，代码如图 5.29 所示。

```
if( t_nCurrent > pFaultValue->wStallCurMax ) //电机电流大于堵转最大电流
{
    pFaultValue->nStallCurFlag = 1;
}
else
{
    if( t_nCurrent < pFaultValue->wStallCurMin ) //电机电流小于堵转消除电流
    {
```

```

        pFaultValue->nStallCurFlag = 0;
    }
}

```

图 5.29 堵转检测方式二

➤ 缺相检测 FaultPhaseCheck(STR_MACtrProc *pObj)

电机发生缺相时，三相电流是不对称的。因此可以通过在程序中检测一定时间内的三相电流的最大值，判断三相电流的最大值是否有不对称的情况来实现缺相保护。判断方式如下：

1.三相电流幅值相差 3 倍，则判定为电机缺相，检测时间和检测倍数根据实际修改，程序如所示。

2.有些应用方案在缺相时，由于缺的那一相会有毛刺的存在，可能会导致采集的最大电流值跟另外两相差不多，这时候通过上述方法可能检测不出来。解决方法：一定时间内电流幅值积分小于设定阈值，则判定为缺相。程序如图 5.30 所示：

```

if(((pstruImaxCurrent->nPhaseU >> 2) > pstruImaxCurrent->nPhaseV) || (((pstruImaxCurrent->nPhaseV >> 2) > pstruImaxCurrent->nPhaseW)) ||
    (((pstruImaxCurrent->nPhaseW >> 2) > pstruImaxCurrent->nPhaseU))) /*如果三相电流幅值相差 3 倍
则判定为缺相，倍数需要根据实际负载调整*/
{
    pObj->m_pFaults->B.PhaseLossError          = 1;          /* 缺相错误标志位置 1 */
    pObj->m_pFaultValue->nPhaseLossFlag        = 1;          /* 第一种缺相保护 */
}

```

图 5.30 缺相保护方式一

```

if((pstruCurrentAmplitude->nPhA < pObj->m_pFaultValue->nPhaseLossCurrent) /* 三相电流幅值小于
设定阈值，累积时间小于设定阈值则判断为缺相*/
    || (pstruCurrentAmplitude->nPhB < pObj->m_pFaultValue->nPhaseLossCurrent)
    || (pstruCurrentAmplitude->nPhC < pObj->m_pFaultValue->nPhaseLossCurrent))
{
    if(pstruFaultTime->nCheckCnt2 < pDrvCfgPara->mS_FBEmptyStall.nLossPHDetectTime) /*检测时间为
pDrvCfgPara->mS_FBEmptyStall.nLossPHDetectTime * 5ms*/
    {
        pstruFaultTime->nCheckCnt2 += FAULT_DETECT_CYCLE;
    }
    else
    {
        pstruFaultTime->nCheckCnt2          = 0;
        pObj->m_pFaults->B.PhaseLossError    = 1;          /* 缺相错误标志位置 1 */
        pObj->m_pFaultValue->nPhaseLossFlag  = 2;          /*第二种缺相保护 */
    }
}

```

图 5.31 缺相保护方式二

前期调试阶段失能故障恢复、故障恢复时间加长，出现运行异常时要能看到什么故障先开启过流保护、过欠压保护，其它保护暂时关闭，对应的设置既可以在 **lksmotor** 上位机中进行设置，也可以在代码对应的宏定义处修改，这里以上位机 **lksmotor** 为主

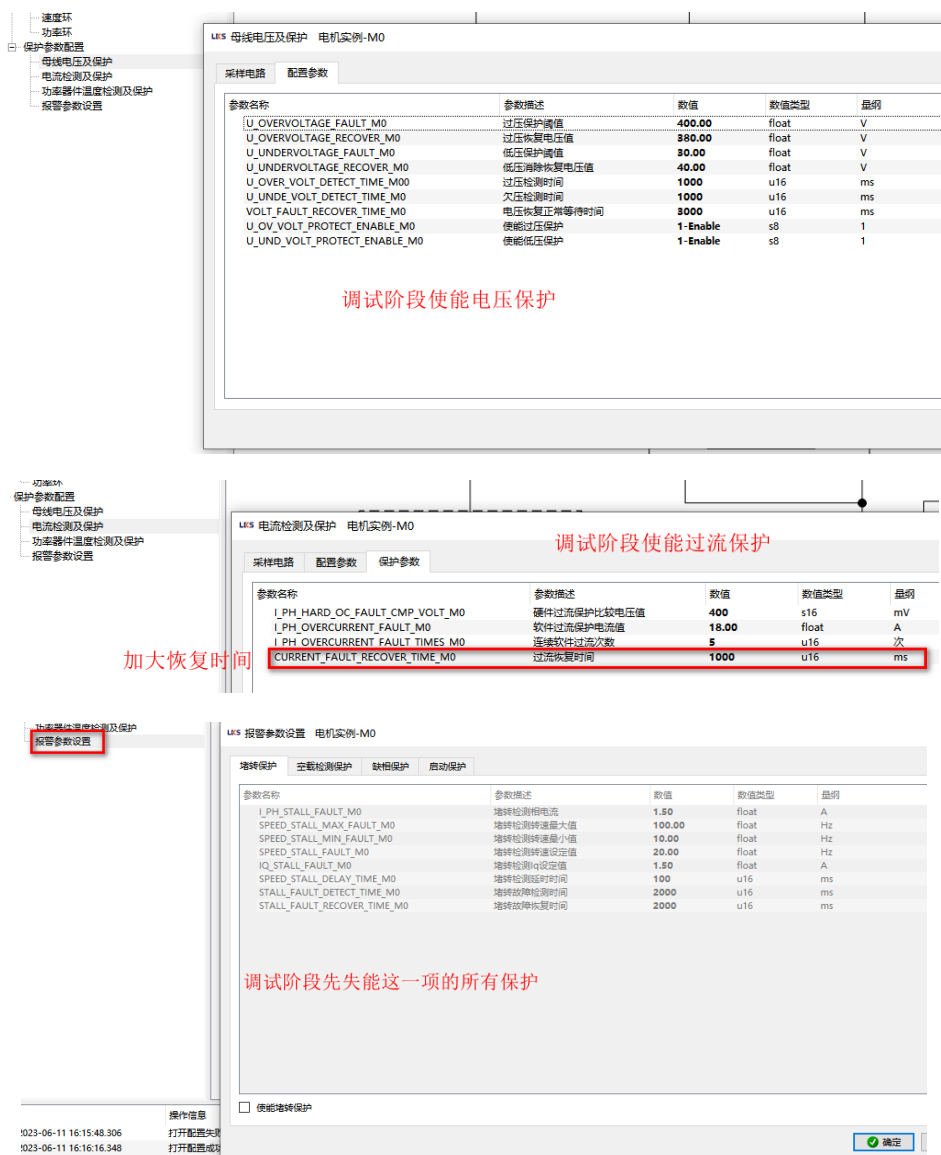


图 5.32 故障保护设置



图 5.33 故障恢复使能

5.7 低压压保护异常处理

电机运行过程中出现异常，如掉电或者母线电压变低等异常时，库中已经包含了进行电机控制和母线电压维持的处理函数，用户可以编入相应代码实现所需的功能。电压保护如下，下图 5.27 中左图为运行中电压丢失，右图为运行中电压丢失然后恢复，具体在程序中修改的参数位置在 MC_Parameter_M0.h 图 5.28 所示

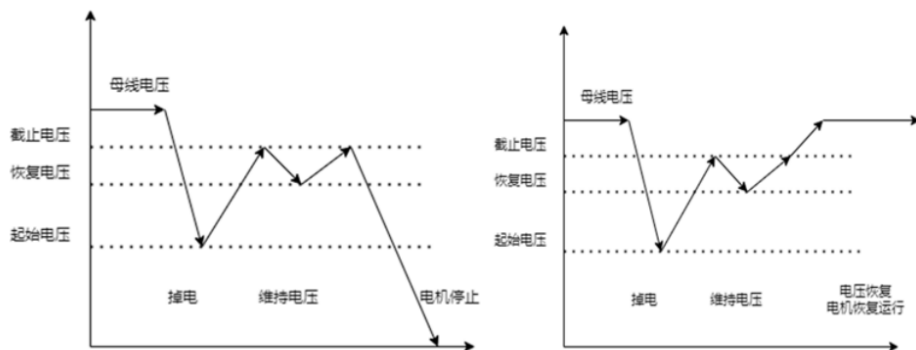


图 5.27 母线电压保护

#define U_UNDVOLT_PROT_ENABLE_M0	(0)		
#define U_UNDVOLT_START_M0	(16)	/* 单位: V 欠压恢复设定值 */	/* AC175 */
#define U_UNDVOLT_STOP_M0	(20)	/* 单位: V 欠压恢复设定值 */	/* AC175 */
#define U_UNDVOLT_RECOVER_M0	(22)	/* 单位: V 欠压恢复设定值 */	/* AC160 */
#define IQ_BRAKE_SET_M0	(1.5)	/* 单位: A IqRef, Iq 制动给定值 */	

图 5.28 母线电压保护设置

5.8 弱磁控制

自动弱磁功能可以通过编译宏决定是否编译代码，通过编译宏编译后，可以设置最大弱磁电流数值决定弱磁是否起作用（设置最大弱磁电流为 0 时，弱磁功能实际不起作用）。对于弱磁功能要求严格的，例如在当前电压下达到最大弱磁扭矩输出，后续实现动态最大弱磁电流调整功能。弱磁参数的调整及设置需要根据实际应用进行调试，修改的地方在 MC_Parameter_M0.h 的 AUTO_FW_LIM_M0

#define WEAKEN_FLUX_STATUS_M0	(TRUE)	/* 弱磁控制状态, TRUE = 使能, FALSE = 不使能 */
#define AUTO_FW_LIM_M0	(0.00)	/* 自动弱磁 D 轴电流限制 A */

图 5.29 弱磁设置

5.9 lkscope 常用调试变量

用户通过 lksmotor 建立工程，单击 lksmotor 中的保存配置，会同时在工程目录下生成对应的 lkscope 调试工程，该工程提供了在电机调试中常用到的一些变量，具体 lkscope 调试的变量说明如下，图中的可查看的数值变量已经在工程中配置完成，用户可以根据自己的需

要进行，每个数值串口与右下角底栏的相对应，对于不需要获取的变量，用户可以取消选中，以提高数据的获取速率。

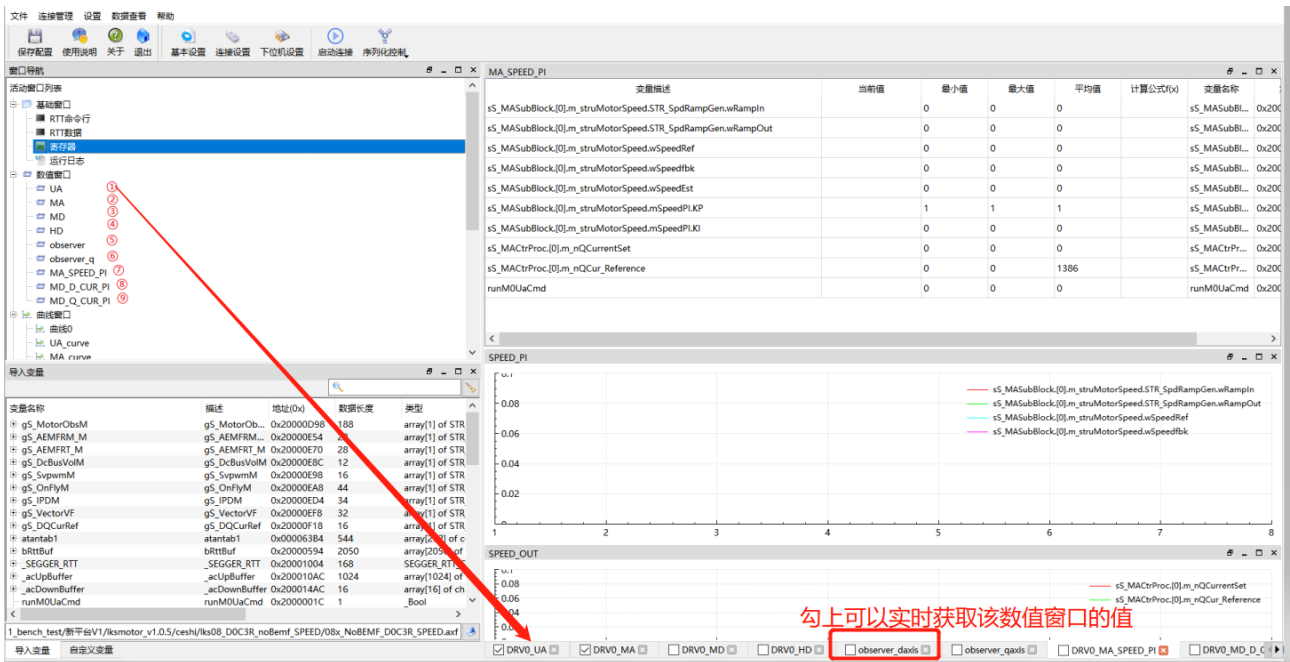


图 5-30 调试界面

根据测试例程文件中的 Ikscope 配置工程，将分别对①-⑨中几个关键的调试变量进行简单的介绍，没有介绍的变量，用户可以根据自己的芯片系列以及具体需要，在调试中进行熟悉，具体也可以向凌鸥官方咨询。下面是对软件中相关数值窗口中的一些常用变量的介绍，文档是对一些程序中的基本变量进行介绍，具体实际的 Ikscope 中配置的变量以实际 Ikscope 中为主，如果有相关不熟悉的变量，可以与凌鸥官方进行沟通。

5.9.1 电机启停、力矩、转速调节

①: UA 窗口;UserApp 层，用户可以更改电机运行命令，以及相关目标电流，速度值，功率值等，例如用户可以实时更改运行命令变量和目标速度在值进行电机启停和转速的调节

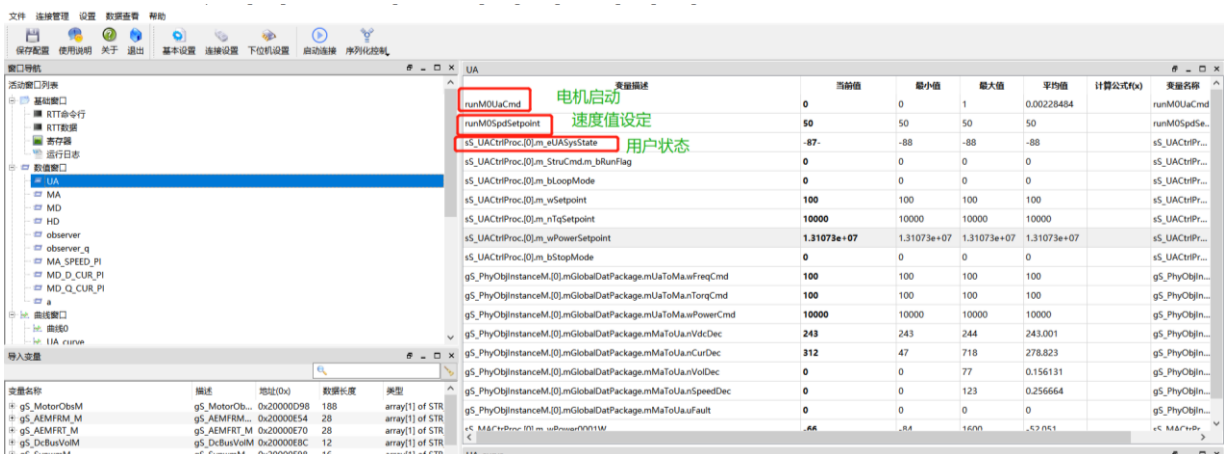


图 5-31：控制运行标志

UA 标签页的相关变量含义

UA	
变量描述	
runM0UaCmd	启停命令
runM0SpdSetpoint	速度设定值
sS_UActrlProc.[0].m_eUASysState	UA层运行状态机
sS_UActrlProc.[0].m_StruCnd.m_bRunFlag	用户控制命令字
sS_UActrlProc.[0].m_bLoopMode	当前环路
sS_UActrlProc.[0].m_wSetpoint	速度设置点 (0.01hz)
sS_UActrlProc.[0].m_nTqSetpoint	力矩设置点 0.001A
sS_UActrlProc.[0].m_wPowerSetpoint	功率设置点 0.01w
sS_UActrlProc.[0].m_bStopMode	停止模式
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wFreqCmd	电频率设定值
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.nTorqCmd	力矩值设定值
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mUaToMa.wPowerC...	功率值设定值
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.nVdcDec	母线电压值(0.1)
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.nCurDec	电流峰峰值
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.nVolDec	电压值 $\sqrt{u_d^2 + u_q^2}$
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.nSpeedDec	当前电频率
gS_PhyObjInstanceM.[0].mGlobalDatPackage.mMaToUa.uFault	当前运行故障
sS_MACtrProc.[0].m_wPower0001W	当前功率值

5.9.2 保护标志，如过流、过压等保护

②：MA 窗口;MotorDrive 层：查看电机运行过程中的故障异常种类，都会实时更新到该故障标志当中，2 个应用层的错误保护标志是一致的，UA 层是 MA 错误标志的总集

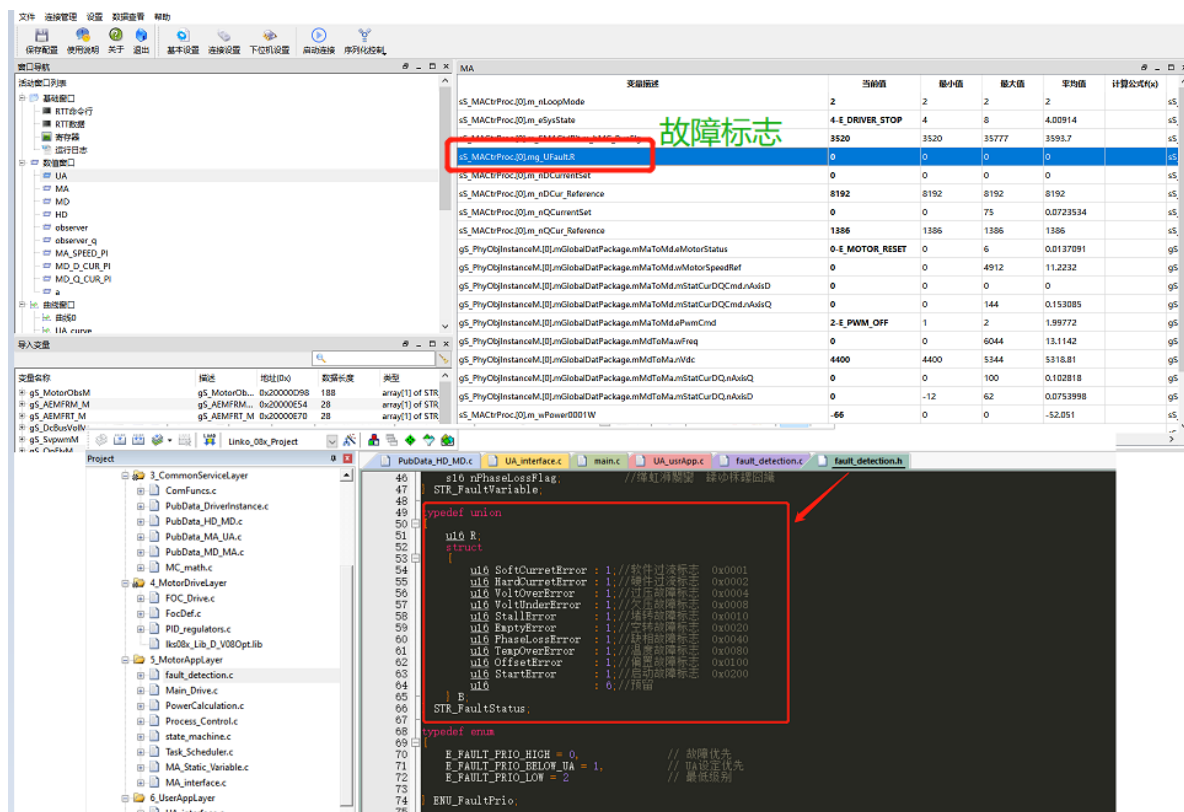


图 5-32: 查看 MA 电机应用层错误标志

MA 层相关变量含义

sS_MACtrProc[0].m_nLoopMode	当前运行环路
sS_MACtrProc[0].m_eSysState	电机当前状态
sS_MACtrProc[0].m_SMACtrBit.m_bMC_RunFig	电机运行控制字域
sS_MACtrProc[0].mg_UFault.R	电机当前故障状态
sS_MACtrProc[0].m_nDCurrentSet	Id设定值
sS_MACtrProc[0].m_nDCur_Reference	Id参考值
sS_MACtrProc[0].m_nQCurrentSet	iq设定值
sS_MACtrProc[0].m_nQCur_Reference	iq参考值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToMd.eMotorStatus	电机当前状态 (Focdrive.c)
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToMd.wMotorSpeedRef	速度参考值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToMd.mStatCurDQCmd.n...	Q轴给定值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToMd.mStatCurDQCmd.n...	D轴给定值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMaToMd.ePwmCmd	PWM使能命令
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMdToMa.wFreq	电频率值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMdToMa.nVdc	母线电压采集值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMdToMa.mStatCurDQ.nAxisQ	Q轴反馈值
gS_PhyObjInstanceM[0].mGlobalDatPackage.mMdToMa.mStatCurDQ.nAxisD	D轴反馈值
sS_MACtrProc[0].m_wPower0001W	功率值
sS_MASubBlock[0].m_struMotorSpeed.wSpeedRef	速度参考值
sS_MASubBlock[0].m_struMotorSpeed.wSpeedSet	速度设定值
sS_MASubBlock[0].m_struMotorSpeed.STR_SpdRampGen.wRampIn	速度斜坡输入

5.9.3 系统状态机查看

状态机有 2 个，分别为①：UA 窗口：用户应用层(UA)-低频任务 1ms 周期，一般只

需关注这个即可；②：MA 窗口：电机应用层(MA)-高频任务 MCPWM 周期，想深入了解控制算法的可以查看这个层，不建议修改里面的代码；2 个状态机的状态是保持一致的

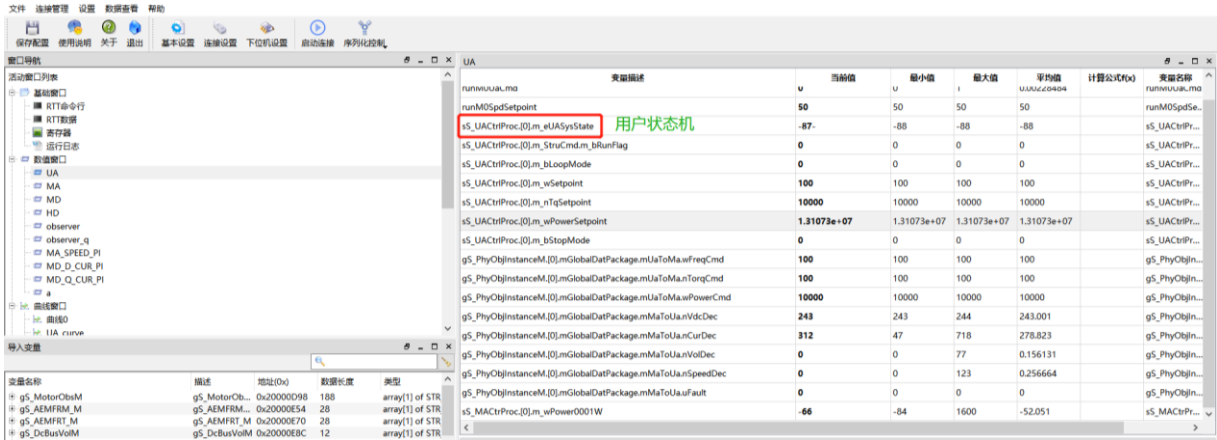


图 5-33：用户应用层状态机查看

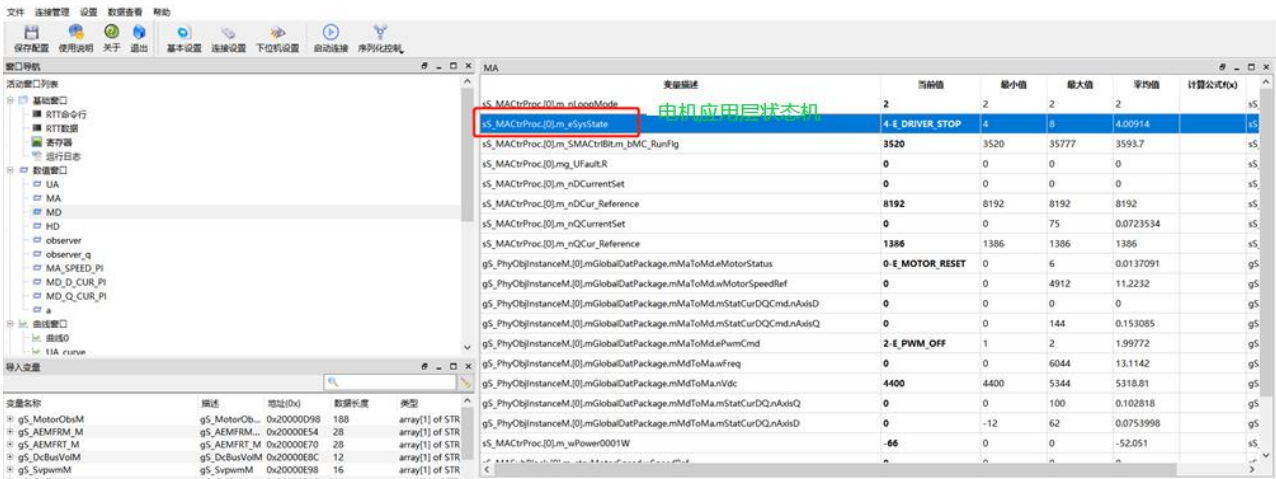


图 5-34：电机应用层状态机查看

5.9.4 占空比与采样

④：HD 串口，用户该窗口查看 PWM 输出值，偏移值等：

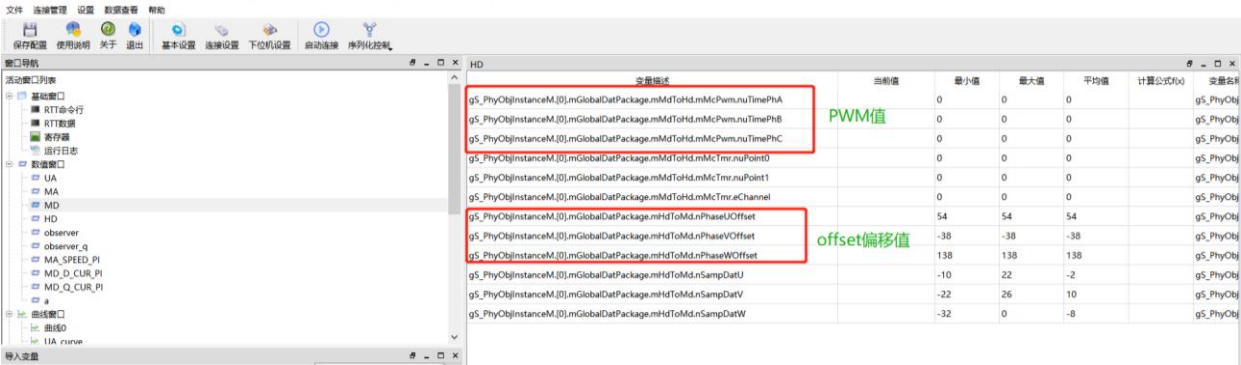


图 5-35：硬件驱动层变量查看

5.9.5 观测器的输出

⑤、⑥：observer 窗口：查看调试观测器 d, q 轴的观测器的反电动势的值

变量描述	当前值	最小值	最大值	平均值	计算公式(x)	变量名称	地址	显示方式
g5_MotorObsM[0].mFluxObsVar.mFasUd	138	138	138			g5_MotorObs...	0x20000d42	Signed
g5_MotorObsM[0].mFluxObsVar.mFasUq	30117	30117	30117			g5_MotorObs...	0x20000d44	Signed
g5_MotorObsM[0].mFluxObsVar.mFasI	9750	9750	9750			g5_MotorObs...	0x20000d4c	Signed
g5_MotorObsM[0].mFluxObsVar.mFasIqTest	0	0	0			g5_MotorObs...	0x20000d4e	Signed
g5_MotorObsM[0].mFluxObsVar.mFasIdEstN_1_Fs	0	0	0		x^4/65536	g5_MotorObs...	0x20000d4f	Signed
g5_MotorObsM[0].mFluxObsVar.mFasIqEstN_1_Fs	0.0625	0.0625	0.0625		x^4/65536	g5_MotorObs...	0x20000d48	Signed

图 5-36 反电动势观测值查看

5.9.6 速度反馈与 PI 调节

⑦：MA_SPEED_PI 窗口：用户可以调试速度环的 KP, KI 达到理想的收敛效果

变量描述	当前值	最小值	最大值	平均值	计算公式(x)	变量名称	地址	显示方式
s5_MASubBlock[0].m_struMotorSpeed.STR_SpdRampGen.wRampIn	0	0	0	0		s5_MASubBl...	0x20000d42	Signed
s5_MASubBlock[0].m_struMotorSpeed.STR_SpdRampGen.wRampOut	0	0	0	0		s5_MASubBl...	0x20000d44	Signed
s5_MASubBlock[0].m_struMotorSpeed.wSpeedRef	0	0	0	0		s5_MASubBl...	0x20000d4c	Signed
s5_MASubBlock[0].m_struMotorSpeed.wSpeedFbk	0	0	0	0		s5_MASubBl...	0x20000d4e	Signed
s5_MASubBlock[0].m_struMotorSpeed.wSpeedEst	0	0	0	0		s5_MASubBl...	0x20000d4f	Signed
s5_MASubBlock[0].m_struMotorSpeed.mSpeedPLKP	1	1	1	1		s5_MASubBl...	0x20000d50	Signed
s5_MASubBlock[0].m_struMotorSpeed.mSpeedPLKI	0	0	0	0		s5_MASubBl...	0x20000d51	Signed
s5_MACtrProc[0].m_nQCurrentSet	0	0	0	0		s5_MACtrPr...	0x20000d52	Signed
s5_MACtrProc[0].m_nQCur_Reference	0	0	1386			s5_MACtrPr...	0x20000d53	Signed
runM0UaCmd	0	0	0	0		runM0UaCmd	0x20000d54	Signed

图 5-37：速度环变量调试

5.9.7 电流反馈与 PI 调节

⑧、⑨：MD_D/Q_CUR_PI 窗口：电流环 q, d 轴的反馈与 PI 实时调节

变量描述	当前值	最小值	最大值	平均值	计算公式(x)	变量名称	地址	显示方式
g5_FocObjM[0].m_pStatCurrDQRef->nAxisD	0	0	0	0		g5_FocObjM...	0x20000220	Signed
g5_FocObjM[0].m_pStatCurrDQ->nAxisD	0	0	0	0		g5_FocObjM...	0x2000021c	Signed
g5_FocElementObjM[0].mAcroD.KP	0	0	1606			g5_FocEleme...	0x20000288	Unsigned
g5_FocElementObjM[0].mAcroD.KI	0	0	1505			g5_FocEleme...	0x2000028a	Unsigned
g5_FocElementObjM[0].mAcroD.mAcroDOut	0	0	0	0		g5_FocEleme...	0x20000296	Signed
g5_FocElementObjM[0].mAcroDQ.nAxisD	0	0	0	0		g5_FocEleme...	0x200002ba	Signed
g5_MotorObsM[0].mFluxObsVar.s16UmRohRef	8192	8192	8192			g5_MotorObs...	0x20000dc6	Signed

图 5-38：电流环变量调试

5.9.8 算法状态变量

高速数据：相电流、强拖角度/观测器角度：相电流、强拖角度/观测器角度在 ADC 中断服务函数或 MCPWM 中断服务函数，配置 RTT 需要发送的数据

6.常见异常处理

6.1 错误保护标志

电机启动后，没任何反应，请查看电压保护标志、电流保护标志等；确认电压、电流保护参数设置合适

lksscope 错误标志查看，在搜索框输入 stru_Faults 选择 R 并拖拽到数值窗口中(具体保护标志可拖拽下面的位域变量成员)，此结构体变量属于电机应用层 MA，官方提供的 lksscope 中已经添加了该变量，具体可以参考第 5 章 lksscope 常见变量调试

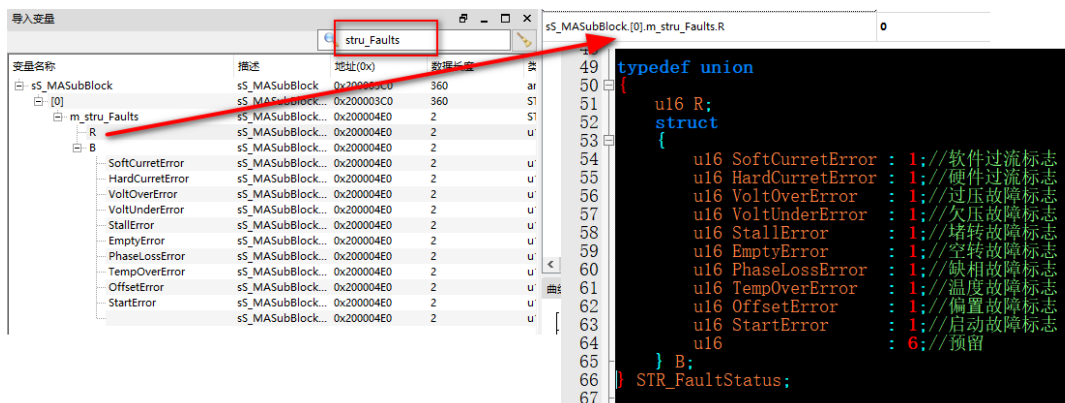


图 6-1：查看错误保护标志

6.2 启动电流大

电机启动瞬间有一个大的电流导致过流保护、这个现象可以通过示波器测试相电流或通过 lksscope 查看错误保护标志可知，可能 2 个原因导致(前提是 lksmotor 参数配置和硬件一致)：

<2>相电流/母线电流 ADC 通道软件配置和硬件不一致

<1>MCPWM 3 对 PWM 软件配置和实际硬件 3 相的电路不一致

启动时有很大的相电流(相电流峰值达到十几 A)，一般会触发硬件或软件过流报警，电机不能正常运行，如果确认 lksmotor 参数配置没有错误，大概率是下位机 MCU 软件和硬件不匹配，可以对照官方的 demo 板套件 和 自己的 MCU+底板 进行对照，进而调整硬件配置，对照下图(这个后面针对不同芯片系列信号做具体的说明，以供参考)

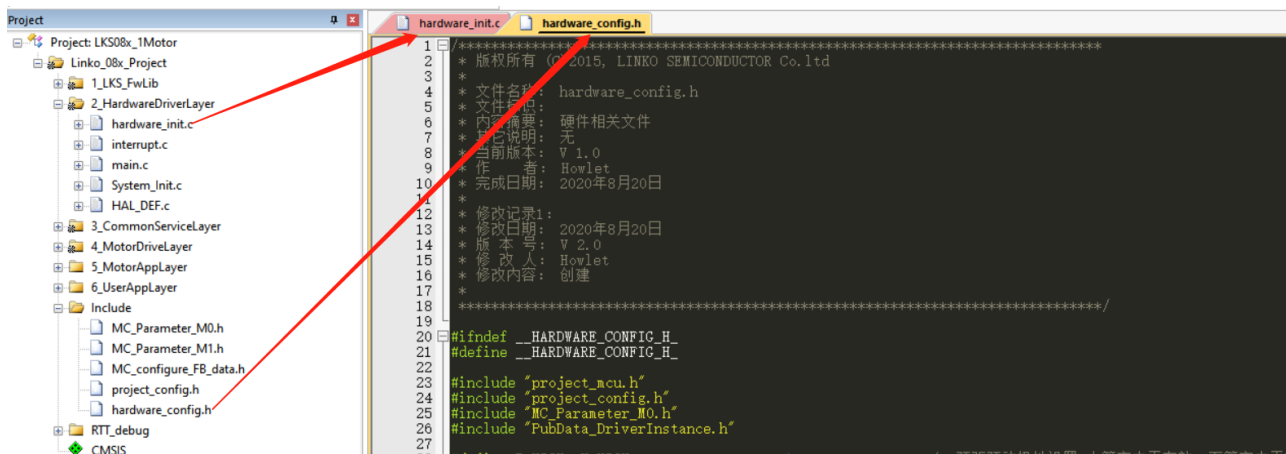


图 6-2: MCU 外配置

6.3 明显抖动

不是正常状态且没有切入到闭环，这个大概是初始位识别偏差大，强拖失败的表现；
解决方法：调整强拖时的电流加减值、目标电流大小、强拖速度加减值

6.4 停机检测失败

在下位机 MCU 端通过 KEIL 烧写程序后，然后首次用 lksmotor 更新配置参数，会提示如下信息，这里选择【跳过并继续】

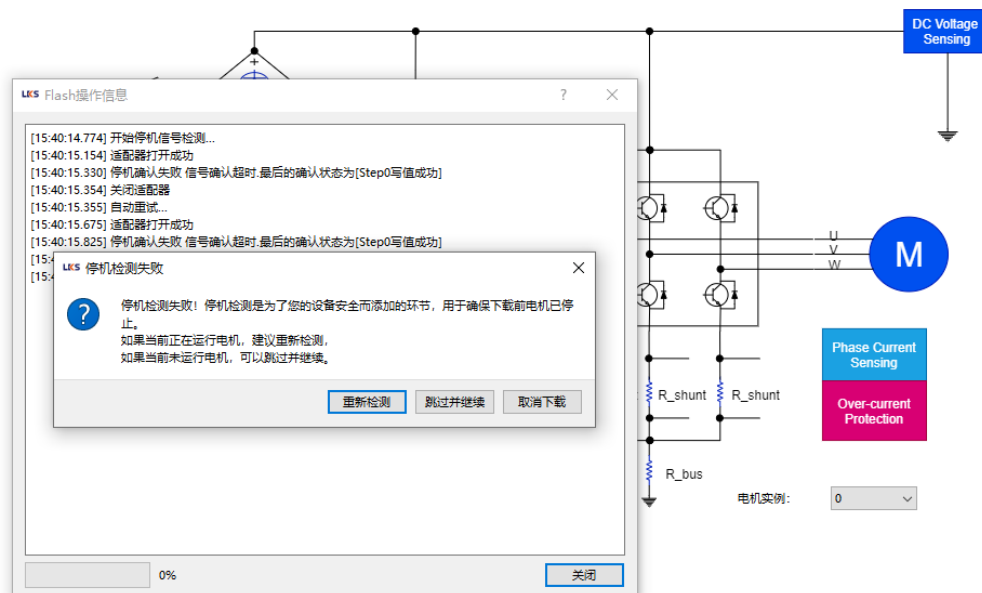


图 6-3: 停机检测失败提示

这是因为 MCU 通过 KEIL 烧写程序后，没有复位

解决办法：勾选 KEIL 中 Reset and Run 选项 或者 在 KEIL 下载程序后手动掉电/上电复位这样就不再弹出该提示框

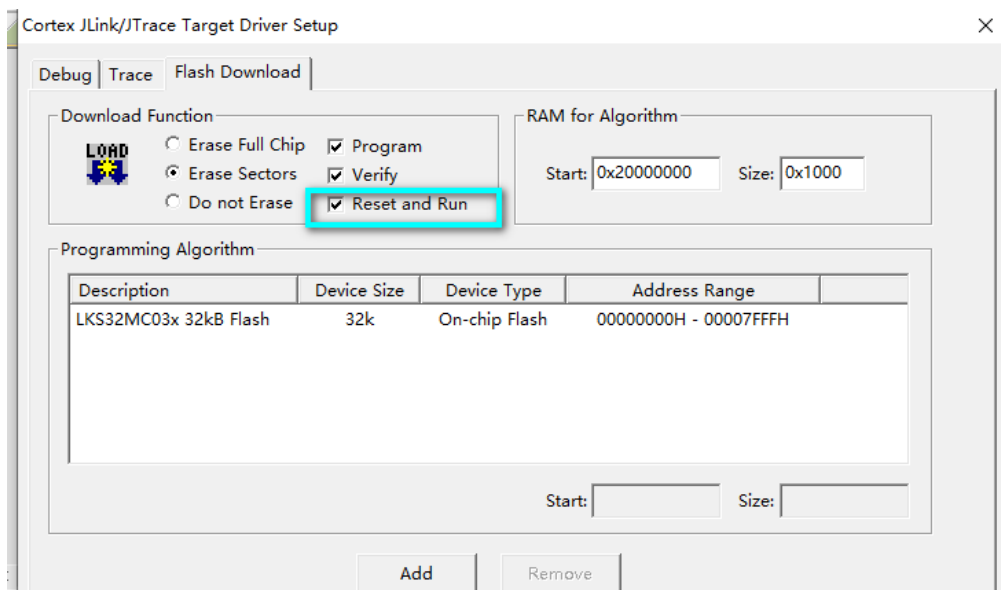


图 6-4: KEIL 勾选复位

另外说明,通过 KEIL 的 Debug 在线硬件调试 和 lksflash 下载程序后不会出现这个现象,因为二者都会复位芯片、lksmotor 下载程序后会触发 MCU 软复位;

6.5 偏置电压读取

会出现没有保护标志置位,但是电机无法控制运行的现象,可能一种情况如下:
新平台是默认顺逆风检测的,检测阈值是比较设定 lksmotor 设定的参数和实际检测的相电流值,如果大于阈值,系统会识别成电机仍在运行,而实际电机是停止状态,可以把阈值提高再实验,目前使用外部电阻做检流,顺逆风检测和偏置读取在程序结构上一致,用读取偏置电压也能反应

参数名称	参数描述	数值	数值类型	量纲
SPEED_TRACK_DELAYTIME_M0	顺逆风检测时间	200	s16	ms
SPEED_TRACK_ON_FREQ_THH_M0	顺风切闭环频率	15.00	float	1Hz
EBRK ON_FREQ_THH_M0	逆风刹车频率	10.00	float	1Hz
MOTOR_STOP_CUR_THD_M0	电机停转电流检测阈值	1.10	float	A
MOTOR_STOP_CUR_DIF_THD_M0	电机停转电流差值检测阈值	1.00	float	A
STOP_TIME_M0	电机停止检测滤波时间	500	s16	ms
STOP_DELAY_TIME_M0	电机停止后延迟时间	500	s16	ms

图 6-4: 停机检测电流阈值设定

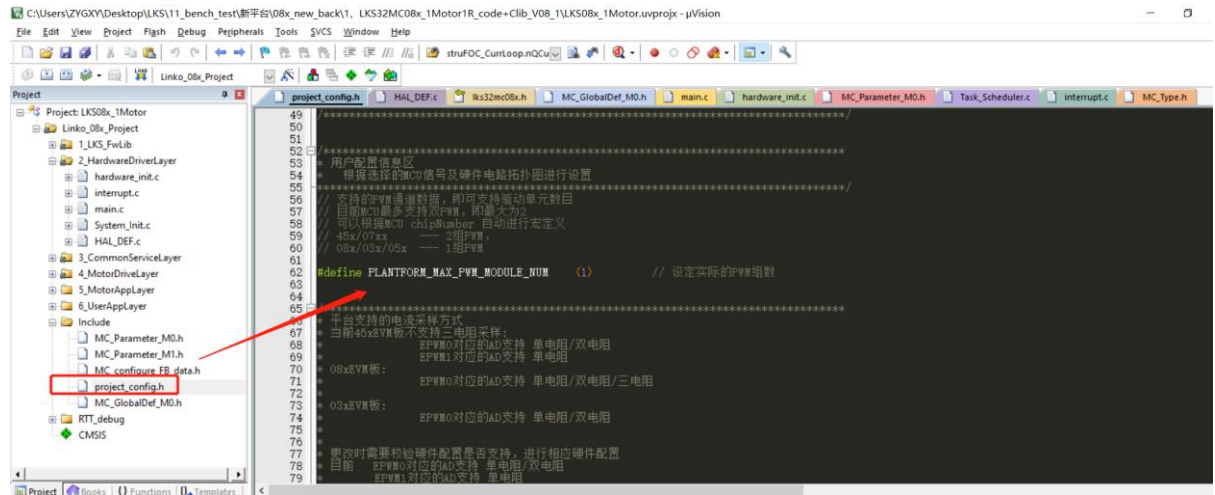
6.6 电机启动运行进入伪收敛状态

电机启动进入速度环,由于负载的突然变化,电机转子停转,但是运行状态显示 Run 状态,这里一般要确认一下电机运行的额定电流,对于速度环的输出限制 I_{qmax} 和 I_{dmax} 是否设定合适,其次调节 k_p , k_i 的值,另外可以在程序中判断速度和频率的比值是否异常,进行

二次启动等。

7.不同硬件之间软件移植

- 用户根据自己的 mcu 和硬件修改底层代码时，首先需要配置 project_config.h 文件，具体文件中对驱动单元有详细的注释，用户可以自行去了解，用户需要根据自己要移植的 mcu 与硬件电路，进行相关宏参数的修改。



- 用户需要根据自己需要移植的 mcu 系列和硬件电路，更改驱动单元的配置文件，这里以 LKS32MC03x 为例,示例程序使用的 MCU 是 LKS32MC037M6S8 + 外置预驱，现在移植到 LKS32MC034DF6Q8、内置预驱；LKS32MC08x 移植方法类似

7.1 硬件说明

硬件：MCU 类型、外围硬件接口、ADC 通道选择

	内置预驱	型号	内置驱动类型
MC03x	是	LKS32MC034xxxx	6N
		LKS32MC035xxxx	3P3N
		LKS32MC037Exxxx	
		LKS32MC037Fxxxx	
		LKS32MC037Qxxxx	
		LKS32MC039Dxxxx	
	否	LKS32MC033xxxx	
		LKS32MC037Mxxx	
		LKS32MC038xxxx	
		LKS32MC039Pxxx	

表 7-1：LKS32MC03x 系列的预区说明

7.2 一般修改参数模块:

OPA/母线电压/电流/MCPWM:

OPA/母线电压/电流按照硬件配置即可，这里重点说明一下 MCPWM 的配置

常见的硬件控制结构如下:内置/外置预区、三相半桥结构

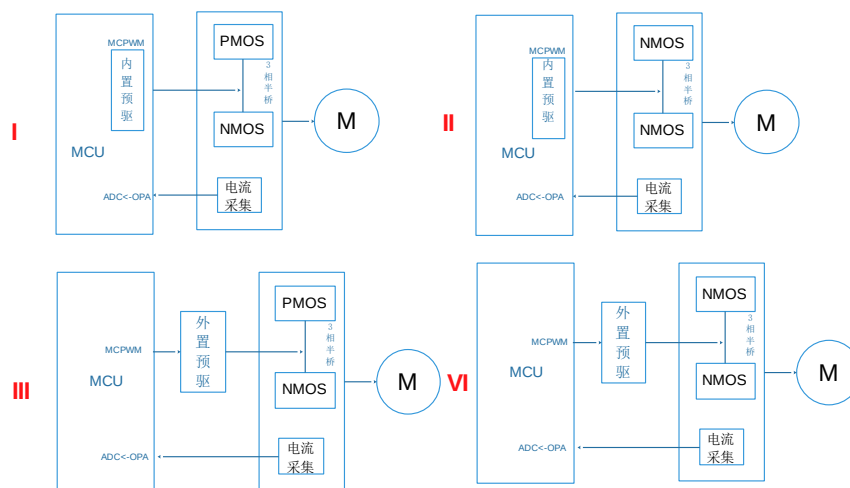


图 7-1：常见控制电路结构

7.3 驱动极性分析

示例代码采用 **III** 结构，MC037M6S8+预区 LKS560，选择 MCPWM 驱动极性是 P_HIGH_N_LOW；而移植到 MC034DF6Q8 是内置预区，采用 **I** 结构，MCPWM 驱动极性是 P_HIGH_N_HIGH，MC03x 不同型号的移植涉及到如下 2 个文件：hardware_init.c、hardware_config.h，这 2 个文件配置是和硬件紧密相关的，根据硬件的接口设计来对照修改相关程序

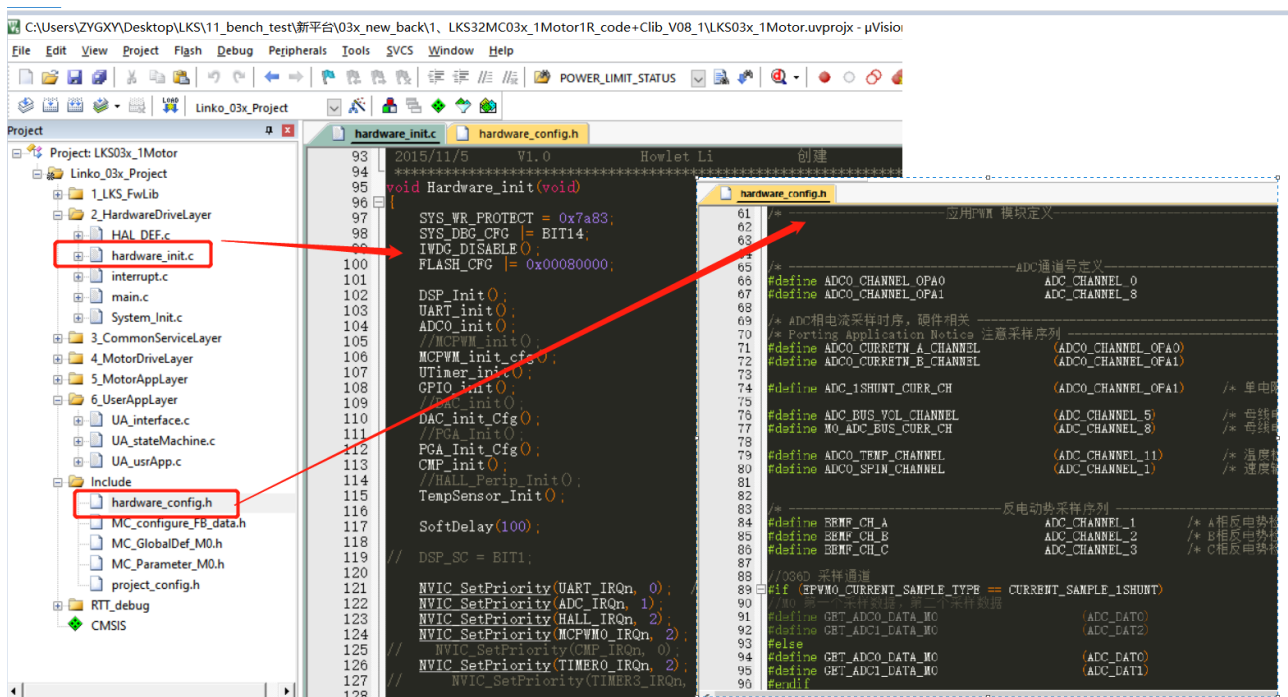


图 7-2：初始化 MCU 基本外设

hardware_init.c: 硬件接口配置 MCPWM/ADC/PGA/GPIO/DAC 等

MCPWM: 对于 MC03xx 系列，这里基本配置无需修改，可能在调试时把 FAIL 暂时关闭，Debug 调试输出 PWM 选项使能

ADC: 主要是通道数和通道顺序，这个和硬件相关

PGA: OAP0/OPA1 对应的电机相序，MC034 只支持单/双电阻

GPIO: MCPWM 模块的 IO、有感 HALL 的 IO、其它串口、电位器特定功能

DAC: 主要用来 CMP 的比较参考端，硬件过流保护设置，不对外输出

hardware_config.h: 芯片信号选择、相电流 ADC 通道配置、母线电流/电压 ADC 通道配置

MCU 型号选择: MC03x 分内置预驱 (6N、3P3N)、不带预驱

相电流 ADC 通道配置: 采样 2 相电流/母线电流这个和电机采集相序有关

母线电流/电压 ADC 通道配置: 母线电流是采用 OPA1 进行采集、母线电压根据具体电路决定

7.4 移植步骤

(1) 型号选择、MCPWM 输出有效极性选择

在 **hardware_config.h** 文件中

#define P_HIGH__N_HIGH	0	/* 预驱预动极性设置 上管高电平有效，下管高电平有效
*/		
#define P_HIGH__N_LOW	1	/* 预驱预动极性设置 上管高电平有效，下管低电平有效
*/		


```

#define CHIP_PART_NUMBER          LKS32MC081          /* 芯片型号选择，选择不正确将影响芯片模块的初始化
*/

#if ((CHIP_PART_NUMBER == LKS32MC084D) || (CHIP_PART_NUMBER == LKS32MC086D) || (CHIP_PART_NUMBER ==
LKS32MC087D))
#define MCPWM_SWAP_FUNCTION          1
#define PRE_DRIVER_POLARITY          P_HIGH__N_HIGH /* 预驱预动极性设置 上管高电平有效，下管高电平有效
*/
#else
#define PRE_DRIVER_POLARITY          P_HIGH__N_LOW /* 预驱预动极性设置 上管高电平有效，下管低电平有效 */
#endif

```

图 7-3: MCU 型号选择及有效极性选择

如果存在的信号没有包含自己使用的型号，可根据对应型号的手册（03x_3P3N_DS、03x_6N_DS、03x_DS）来添加

(2)MCPWM 比较值输出通道确定

一般来说 MCU 的 MCPWM 输出通道和电机相线对应关系：CH0->U 相、CH1->V 相、CH2->W 相实际却有区别，比如不带预驱的 Demo 板底板的对应关系是：CH0->W 相、CH1->V 相、CH2->U 相,需要根据实际的区别修改比较值的更新顺序

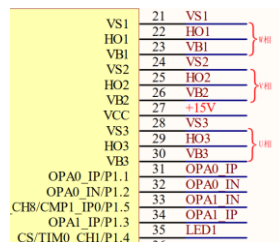


图 7-4: 3 对 PWM 和电机相序对照

```

static inline void HAL_DRV0 MCPWM RegUpdate(STR PubDatMDHD* tS pMdToHd)
{
    MCPWM_TH00 = tS_pMdToHd->mMcPwm.nuTimePhBN;
    MCPWM_TH01 = tS_pMdToHd->mMcPwm.nuTimePhB;

    MCPWM_TH10 = tS_pMdToHd->mMcPwm.nuTimePhAN;
    MCPWM_TH11 = tS_pMdToHd->mMcPwm.nuTimePhA;

    MCPWM_TH20 = tS_pMdToHd->mMcPwm.nuTimePhCN;
    MCPWM_TH21 = tS_pMdToHd->mMcPwm.nuTimePhC;

    MCPWM_TMR0 = tS_pMdToHd->mMcTmr.nuPoint0; // 第一个采样点
    MCPWM_TMR1 = tS_pMdToHd->mMcTmr.nuPoint1; // 第二个采样点
}

```

图 7-5: 修改 3 对 MCPWM 比较值

(3)ADC 通道确定：MC03x 只支持单/双电阻控制模式

单电阻时，母线电流要采样 2 次，分 2 段进行配置，第 1 次采样在第一段，第 2 次采样

在第二段，至于通道顺序满足足够的采样间隔即可

双电阻时，相电流采样各 1 次，采用 1 段式，配置 2 次采样中间有其它通道保证时间足够使得电流稳定，这些通道顺序和对应 ADC 数据寄存器保持一致，至于其它通道配置（电位器、反电动势）有选择的添加，尤其是双电阻的 ADC 通道选择注意和硬件对应好。

```
void ADC0_init(void)
{
    ADC_InitTypeDef ADC_InitStructure;
    ADC_StructInit(&ADC_InitStructure);

    ADC_InitStructure.Align = ADC_LEFT_ALIGN;          /* ADC 转换结果采用左对齐格式 */
    ADC_InitStructure.Trigger_Cnt = 0;                  /* 发生一次触发事件执行一段转换动作 */
    #if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
        ADC_InitStructure.FirSeg_Ch = 2;                /* 第一段共采样 6 个通道 */
        ADC_InitStructure.SecSeg_Ch = 8;                /* 第二段共采样 0 个通道 */
        ADC_InitStructure.MCPWM_Trigger_En = ADC_MCPWM_T0_TRG | ADC_MCPWM_T1_TRG;; /* 打开 MCPWM_T0
MCPWM T1 硬件触发事件 */
        ADC_InitStructure.Trigger_Mode = ADC_2SEG_TRG;    /* 设置 ADC 转换模式为双段式采样 */
        ADC_InitStructure.IE = ADC_EOS1_IRQ_EN;          /* ADC 第一段采样结束中断使能 */
    #else
        #if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_2SHUNT)
            ADC_InitStructure.FirSeg_Ch = 12;            /* 第一段共采样 6 个通道 */
            ADC_InitStructure.SecSeg_Ch = 0;            /* 第二段共采样 0 个通道 */
            ADC_InitStructure.MCPWM_Trigger_En = ADC_MCPWM_T0_TRG ; /* 打开 MCPWM_T0 MCPWM_T1 硬件触发事件 */
            ADC_InitStructure.Trigger_Mode = ADC_1SEG_TRG; /* 设置 ADC 转换模式为单段式采样 */
            ADC_InitStructure.IE = ADC_EOS0_IRQ_EN;      /* ADC 第一段采样结束中断使能 */
        #else
            #if ((EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_3SHUNT) || (EPWM0_CURRENT_SAMPLE_TYPE ==
CURRENT_SAMPLE_MOSFET))
                ADC_InitStructure.FirSeg_Ch = 12;        /* 第一段共采样 6 个通道 */
                ADC_InitStructure.SecSeg_Ch = 0;        /* 第二段共采样 0 个通道 */
                ADC_InitStructure.MCPWM_Trigger_En = ADC_MCPWM_T0_TRG ; /* 打开 MCPWM_T0 MCPWM_T1 硬件触发事件 */
                ADC_InitStructure.Trigger_Mode = ADC_1SEG_TRG; /* 设置 ADC 转换模式为单段式采样 */
                ADC_InitStructure.IE = ADC_EOS0_IRQ_EN;  /* ADC 第一段采样结束中断使能 */
            #endif
        #endif
    #endif
    #endif
    #endif

    ADC_Init(ADC0, &ADC_InitStructure);

    ADC0_IF |= 0xFF;
    #if (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_1SHUNT)
        ADC_1Shunt_NormalModeCFG();
    #elif (EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_2SHUNT)
        ADC_2Shunt_NormalModeCFG();
    #elif ((EPWM0_CURRENT_SAMPLE_TYPE == CURRENT_SAMPLE_3SHUNT) || (EPWM0_CURRENT_SAMPLE_TYPE ==
CURRENT_SAMPLE_MOSFET))
        ADC_3Shunt_NormalModeCFG();
    #endif
}
```

图 7-6: 单双电阻控制 ADC 初始化

这些 ADC 通道顺序配置和读取 ADC 寄存器的顺序一致即可

```
void ADC_1Shunt_NormalModeCFG(void)
{
    ADC0_CHN0 = ADC_1SHUNT_CURR_CH | (ADC_1SHUNT_CURR_CH<<8);
    ADC0_CHN1 = ADC_1SHUNT_CURR_CH | (ADC_1SHUNT_CURR_CH<<8);
    ADC0_CHN2 = ADC_BUS_VOL_CHANNEL | (ADC0_TEMP_CHANNEL<<8);
    ADC0_CHN3 = ADC0_TEMP_CHANNEL | (BEMF_CH_A<<8);
    ADC0_CHN4 = BEMF_CH_B | (BEMF_CH_C<<8);
    ADC0_CHN5 = ADC0_MCU_TEMP_CHANNEL | (ADC0_UVSS_VOLT_CHANNEL << 8);
}
```

图 7-7: 单电阻电流检测 ADC 通道配置

```
void ADC_2Shunt_NormalModeCFG(void)
{
    ADC0_CHN0 = ADC0_CURRETN_A_CHANNEL | (ADC0_CURRETN_B_CHANNEL << 8);
    ADC0_CHN1 = ADC0_CURRETN_A_CHANNEL | (ADC0_CURRETN_B_CHANNEL << 8);
    ADC0_CHN2 = ADC_BUS_VOL_CHANNEL | (ADC_BUS_VOL_CHANNEL << 8); //
    ADC0_CHN3 = ADC0_TEMP_CHANNEL | (BEMF_CH_A<<8);
    ADC0_CHN4 = BEMF_CH_B | (BEMF_CH_C<<8);
    ADC0_CHN5 = ADC0_MCU_TEMP_CHANNEL | (ADC0_UVSS_VOLT_CHANNEL << 8);
    MCPWM_TMR0 = (s16)(100-PWM_PERIOD_M0); // 100-PWM_PERIOD
}
```

图 7-8: 双电阻电流检测 ADC 通道配置

(4)GPIO 配置

主要是 MCPWM 的配置、HALL(目前有感暂未实现)、其它用户功能(串口等)

```
/* MCPWM P1.4~P1.9 */
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_OUT;
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5 | GPIO_Pin_6 | GPIO_Pin_7 | GPIO_Pin_8 |
GPIO_Pin_9;
GPIO_Init(GPIOI1, &GPIO_InitStruct);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_4, AF3_MCPWM);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_5, AF3_MCPWM);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_6, AF3_MCPWM);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_7, AF3_MCPWM);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_8, AF3_MCPWM);
GPIO_PinAFConfig(GPIOI1, GPIO_PinSource_9, AF3_MCPWM);
```

图 7-11: GPIO 配置

8 数据格式

新代码平台的数据均采样定点数进行运算, 03 系列的 sdk 主要采用 16 位定点数据格式进行定义以减少代码空间提升运行速度。

8.1 标么定义

当前的 MCU 内部集成 12 位 AD 运算放大器, 转化结果推荐统一使用左对齐 方式。

AD 量程设置为 2.4V 时如下图所示，详见 LKS32MC03x_User_Manual。

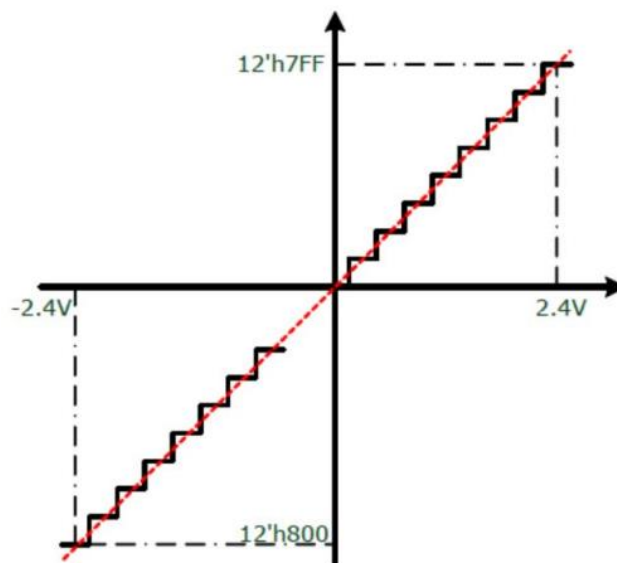
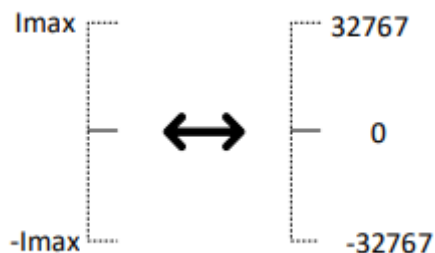


图 8.1 adc 2.4v 量称下的数制

AD 转换结果的数据范围为 $-2^{11} \sim (2^{11}-1)$ ，左移 4 位后范围为 $-32768 \sim 32752$ 。程序中的数据采用 Q15 格式。

1. 电流标幺



电流标幺按照最大采样电流为 I_{base} 进行标幺，具体标幺方案和函数介绍可以参考数据格式及库函数文档，本文档主要介绍，如果对程序中的电流数据转化为实际的物理量，如图 9.2 所示，开环采集到的相电流波形如下：RTT 采集到的电流数据最大值为 718，那么实际的电流值可以它通过 $y/32767 \cdot I_{max}$ 得出，其中 I_{max} 可以根据上位机对应的配置计算看到 I_{max} 的值，从而得到实际的电流值为 899mA

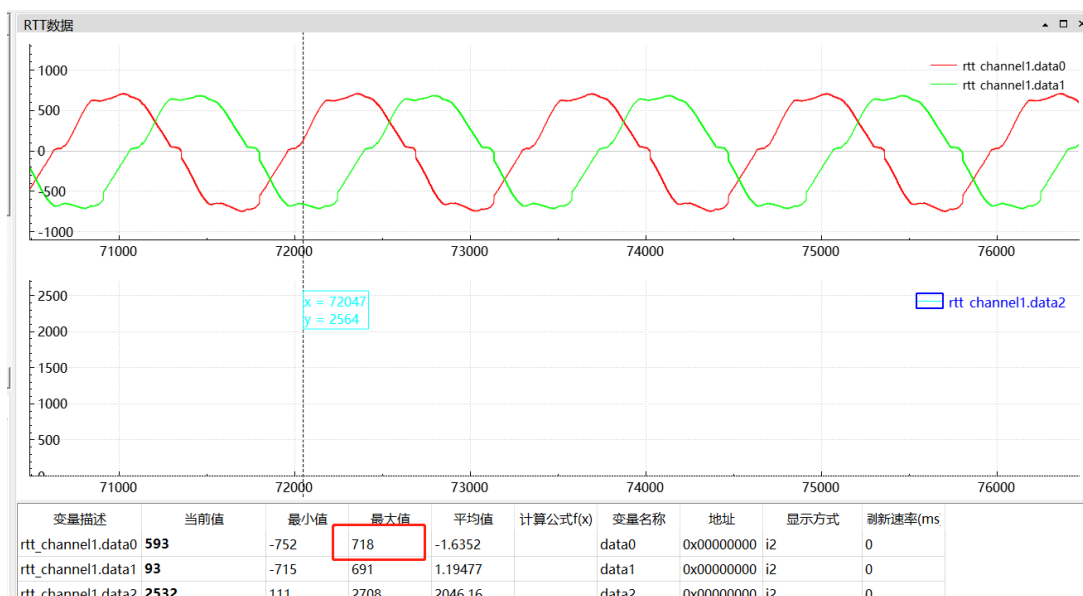


图 8.2 相电流采样

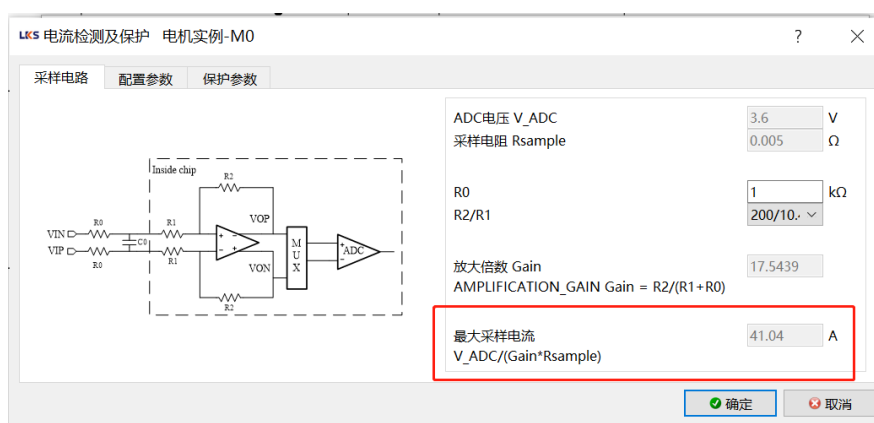
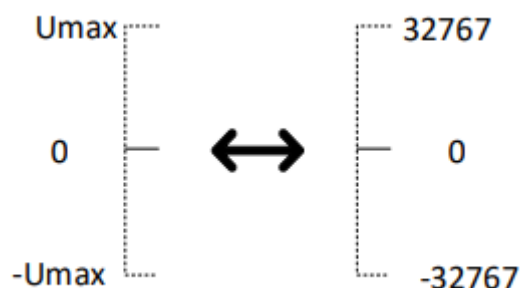


图 8.3 最大采样电流

2. 电压标么



电压标么的转换同电流标么一值，已知电压数据为 y ，实际电压 $U = y / 32767 * U_{max}$ ，其中 U_{max} 为可采集到的最大的电压范围，可以通过硬件配置在 `lksmotor` 中自动计算得出

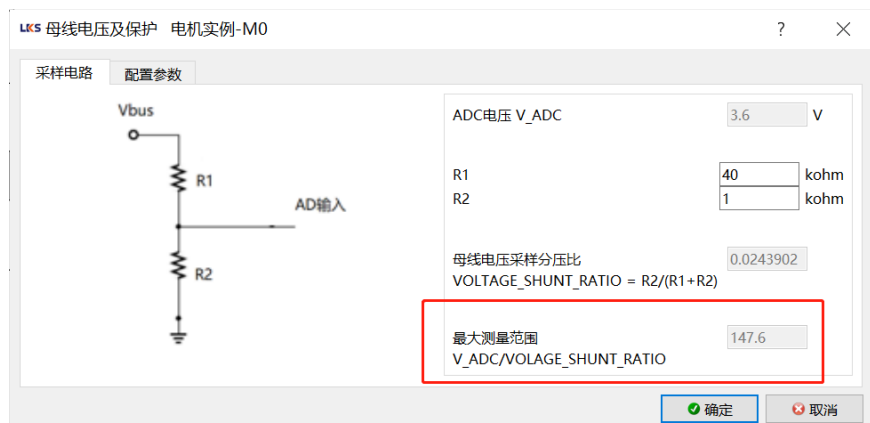
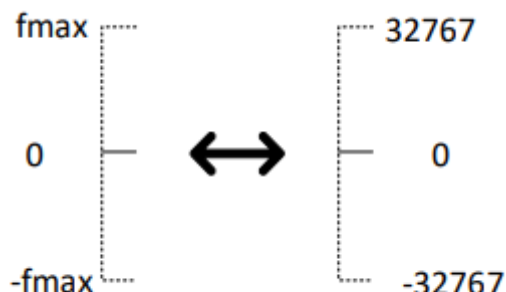


图 8.4 电压最大测量范围

3. 速度标么



基于当前电机运行最高频率，选定合适的最高频率 f_{max} 作为基准值，即 IQ15(1), 然后进行相应的数据标么处理，已知速度数据为 y ，那么实际得速度电频率为 $F=y/32767*f_{max}$ ，其中 f_{max} 为用户设定得电机最高运行电频率

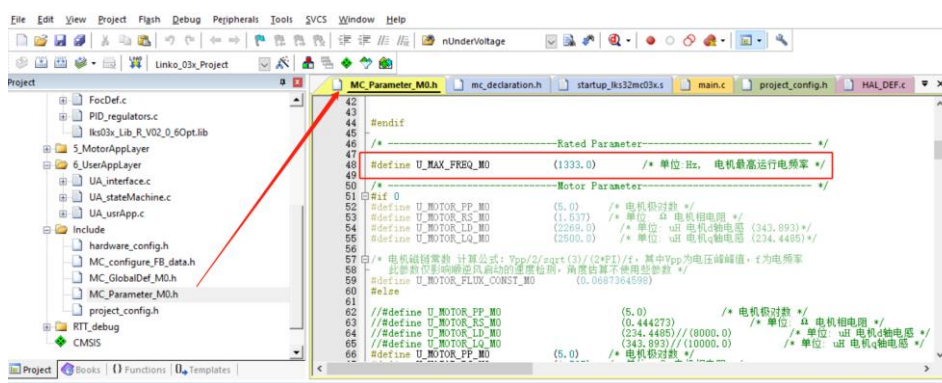
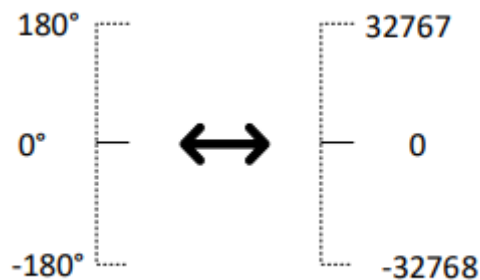


图 8.5 电机运行最高电频率

4. 角度标幺



基于电角度，360°为 65536，进行相应的数据标幺处理，已知角度数据 y ，实际的角度值为 $E=y/32767*180$ 度

8.2 上位机数据单位说明

用户在用上位机调试时，由于数据格式的更改，用户的设定的速度频率，力矩值，功率值得设定都是按照程序中定义的单位进行设置，其中，用户层中频率值得设定单位为 0.01hz,力矩值得设定单位为 0.001A，功率值得设定单位为 0.01w，用户在使用 lksscope 调试时，要注意变量得单位，具体单位的更新按照最新得软件版本为主。

9.新平台控制状态机

9.1 UA 层状态机

UA 层状态机只负责命令的传递，将电机运行状态、刹车方式和错误初始命令下发到 MA 层。UA 层状态机函数 (UA_StateMachine(pObj)) 在 UACtrlObjTask() 函数内,1ms 执行周期。其中 pObj 是 UA 层控制结构体 pSTR_UACtrProc 声明的结构体指针。

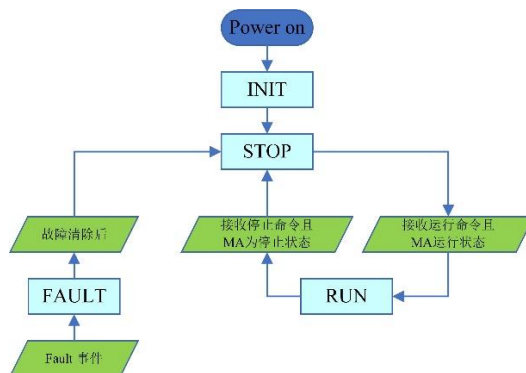


图 9.1 UA 层状态机流程图

UA 层状态机流程图如错误!未找到引用源。所示，以下是图中关键状态和转换的分步概述：

进入 UA_StateMachine(pObj) 后, 先进行故障检测, 如果有故障, 则进入故障状态(E_UA_FAULT); 没故障, 则判断是否上电完成, 如果完成进入状态机, 没完成进入状态机的初始化状态 (E_UA_INIT)。

➤ 初始化状态 (E_UA_INIT)

描述: 目前只检测上电情况。需要参数配置时, 根据应用添加参数配置逻辑处理。

关键程序:

IfPostFinishFromMA(pMA2UA): 判断上电情况。

下一状态: 如果上电完成, 进入停机状态 (E_UA_STOP); 未完成, 则一直在初始化状态。

➤ 停机状态 (E_UA_STOP)

描述: 该状态下电机停止运行, 并等待电机运行指令。

关键程序:

IfUARunCmd(pObj): 判断是否是运行指令。

UARunCmdHandle(pObj): 将运行指令和方式 (如正反转、速度环等) 指令传递到 MA 层。

IsRunningFromMA(pMA2UA): 等待 MA 层回传信号, 确认电机进入运行状态。

下一状态: 当接收到运行命令且 MA 层回传运行状态, 则进入运行状态 (E_UA_RUN)。

➤ 运行状态 (E_UA_RUN)

描述: 该状态下电机正常运行, 并等待停机指令。

关键程序:

IfUAStopCmd(pObj): 判断是否是停止指令。

UAStoppingCmdHandle(pObj): 将停止指令和刹车方式传递到 MA 层。

IsStoppedFromMA(pMA2UA): 等待 MA 层回传信号, 确认电机停止。

下一状态: 当接收到运行命令且 MA 层回传停止状态, 则进入停止状态 (E_UA_STOP)。

➤ 故障状态 (E_UA_FAULT)

描述: 检测到了 MA 层报告的故障或者 UA 层自身故障信号进入故障状态。控制电机停止, 并进行故障恢复。

故障清除逻辑:

如果 UA 层或者 MA 层故障标志存在, 进入 UAFaultStopHandle(pObj)函数, 进行停机。

在 UA 层无故障情况下, MA 层故障解除, 即标志位 pMA2UA->uFault 未置位, 而 pMA2UA->SR.Bit.Fault 置位, 并且使能了故障清除功能(FAULT_AUTO_CLR_M0), 则一段时间后

关键程序：

StopMotorMain(pObj)：负责 MD 层参数初始化和报警处理等，同时检测电机是否停止。

IsMotorRunCmd()：检测是否收到电机运行命令。

下一状态：需要预充电，进入充电状态(E_DRIVER_CHARGE)。如果不需要，则会判断是否需要顺逆风(DRV0_DIR_CHECK_ENABLE)、初始位置检测(DRV0_POS_SEEK_ENABLE)或准备运行状态 (E_DRIVER_READY_RUN)，如 9.3 所示。

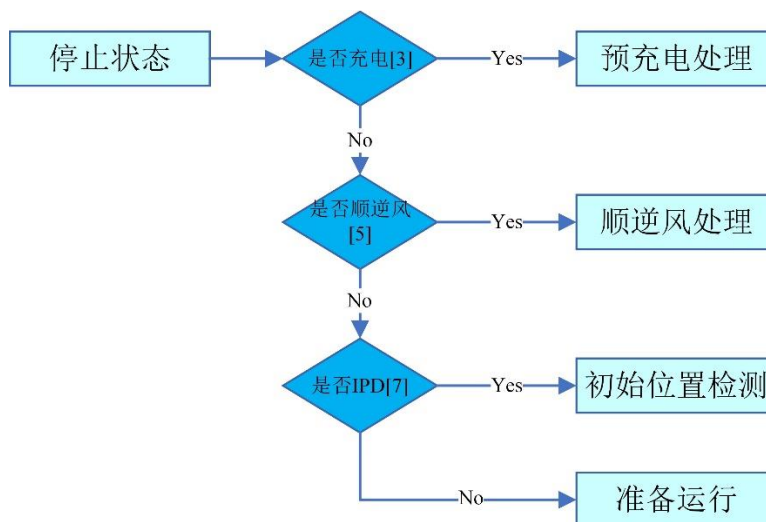


图 9.3 停止状态下，下一状态判断

➤ **充电状态 (E_DRIVER_CHARGE)：**

描述：当上桥自举电容进行预充电时，保证运行时上桥稳定打开。

关键程序：

ChargeDriverMain(pObj)：管理充电过程，设置充电时间，依次打开 A 相、B 相和 C 相下桥进行充电。

IsDriveChargeFinish()：检查预充电过程是否完成。

下一状态：充电完成后，进入**错误!未找到引用源**。顺逆风判断（状态[5]）。

➤ **电机信息检测状态 (E_DRIVER_MOTOR_INFO)：**

描述：中间状态，判断进入顺逆风、初始位置检测还是直接进入运行状态。

关键程序：

MotorTrakingMain(pObj)：顺逆风判断，如果顺风且高于设定的顺风频率，则进入运行状态，其它均进入刹车，也可根据应用自行配置。

MotorIPDMain(pObj)：初始位置检测，将命令由 MA 层转递到 MD 层，在 MD 层进行初始位置检测。

下一个状态：顺逆风后下一状态判断，如 9.4 所示。

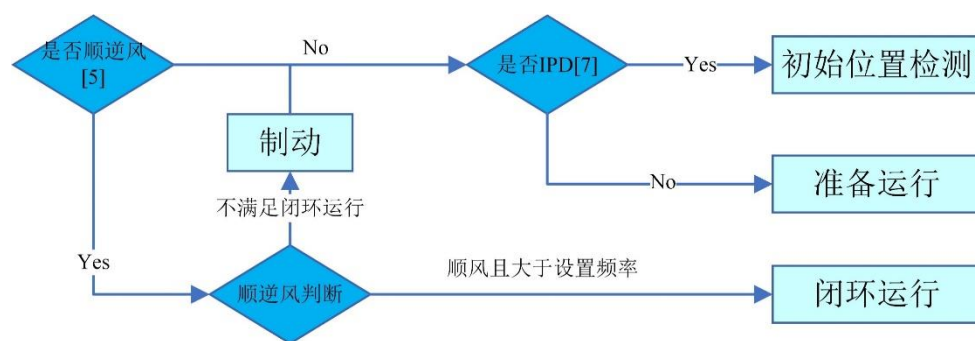


图 9.4 顺逆风后下一状态判断

➤ 准备运行状态 (E_DRIVER_READY_RUN):

描述：中间状态，用来判断是否需要调相，是否有停止命令。

关键程序：

MotorAlignMain (pObj)：对相处理函数。

下一状态：对相结束或者不对相都进入 RUN 运行状态。如果有停止命令则进入停止状态。

➤ 运行状态 (E_DRIVER_RUN):

描述：电机的正常运行状态。在该状态下，电机按照指定的速度和扭矩要求进行运转。将数据传递到 MD 层，驱动电机运行。

关键程序：

MotorCloseMain(pObj)：电机运行控制函数。将 DQ 轴电流、速度、使能 MOE 和运行状态等指令转递到 MD 层。

下一状态：如果接收到停机命令或者运行异常置位，则进入准备停止状态(E_DRIVER_READY_STOP)。

➤ 准备停止状态 (E_DRIVER_READY_STOP)---电机停止过程:

描述：用于在电机从运行状态过渡到停止状态时进行相应的处理，确保电机平稳停止。

关键程序：

MotorBreakMain(pObj)：停机方式来自 UA 层指令。包含三种停机方式：关闭 MOE(MCPWM 输出默认电平)、零电流(Iq_Ref = 0)和下 MOS 开通短接。

IsMotorRunCmd(pObj)：判断电机是否还在运行。

IsCanRestore2Run(pObj)：判断电机是否能够继续运行。

下一状态：如果电机未停止，接收到重启命令，且满足重启条件，进入运行状态(E_DRIVER_RUN)，否则进入停止状态(E_DRIVER_STOP)。

➤ 故障状态 (E_DRIVER_FAULT):

描述: 如果系统检测到任何故障, 系统将进入此状态。该状态的主要功能是处理电机运行中发生的错误或异常情况。

关键逻辑:

软硬件过流、过欠压、堵转等状态(见 STR_FaultStatus 结构体) 会导致电机进入故障状态(E_DRIVER_FAULT)。在该故障状态下, 清除 RUN 标志位(m_bMC_RunFlg= 0), 后进入故障恢复程序(FaultRecover(pObj)), 根据实时信息, 尝试恢复, 同时将信息传递到 MD 和 UA 层。

后续的 IF 判断内将硬件过流故障单独进行处理, 如果不是硬件过流, 则根据报警和上电状态判断进入初始化状态还是停止状态。如果是硬件过流, 并已经恢复, 根据实际情况, 清除 breakIn 故障标志。

关键程序:

MotorFaultMain(pObj): 报警处理函数, 设置 Fault 标志位, 同时将信息通过共享区传递到 MD 和 UA 层。

FaultRecover(pObj): 故障恢复函数, 根据实际采样的信息, 判断故障是否可以清除。

下一状态: IF 判断语句中, 进行错误状态判断和恢复, 如果一直有错误标志, 程序会一直停留在故障状态; 如果无错误标志, 根据上电情况, 进入停止状态(E_DRIVER_STOP)或者初始化状态。

10.补充说明

本文档是对新平台算法软件和利用基础例程开发的介绍手册, 具体文档中对应的软件版本需要以实际的软件为主, 本章是对软件版本更新后, 需要重点补充内容的介绍

1、几种 Flux 方式说明: 新平台程序中有几种观测器方式, 具体用户可以根据实际需要, 选择对应的观测方式, 具体观测器的方式需要以实际的软件为主

```
// 观测器参数初始化
extern void FluxObserveParaCalc(u8 t_bObjId);
//
// 简化代码、低速要求低
extern void PmsmFluxObserveIDSimply(u8 t_bObjId, PSTR_VectorDQ tS_pCurDq);

// 简化代码、低速要求高
extern void PmsmFluxObserveSimply_Full(u8 t_bObjId, PSTR_VectorDQ tS_pCurDq);

// 简化代码、低速要求高, 大范围弱磁
extern void PmsmFluxObserveSimplyWeaken(u8 t_bObjId, PSTR_VectorDQ tS_pCurDq);
```

```

// 低速要求低，执行时间略短
extern void PmsmFluxObserve(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq);

// 低速要求高，执行时间略长
void PmsmFluxObserve_Full(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq);

// 低速要求高，大范围弱磁，执行时间长
void PmsmFluxObserveWeaken(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq);

// 基于 PmsmFluxObserve_Full，堵转处理
void PmsmFluxObserve_Full_En(u8 t_bObjId,PSTR_VectorDQ tS_pCurDq);

```

图 9.1 几种观测方式

- 2、弱磁中的 KP, KI 是手动设置的，需要用接口函数 setFieldWeakenPara，设置控制弱磁的 K P, KI 参数，在弱磁控制时需要调整弱磁控制时根据需要调整弱磁控制比例积分系数
- 3、SVPWM 调制：对双电阻和三电阻，基于执行时间和代码空间提供多种 SVPWM 调制方式，可根据需要选择合适的调制方式，用户可以根据代码控制和执行效率选择对应的调制方式

```

extern void SVPWM_1Shunt(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);
extern void SVPWM_1Shunt_5_7Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //单电阻 5/7 自动切换

extern void SVPWM_1Shunt_En(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //单电阻增强

extern void SVPWM_1Shunt_Mode0(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //单电阻 模式 0
extern void SVPWM_1Shunt_Mode1(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //单电阻 模式 1

extern void SVPWM_2Shunt_En1(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //双电阻增强 1
extern void SVPWM_2Shunt_En2(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //双电阻增强 2
extern void SVPWM_3Shunt(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);
extern void SVPWM_3Shunt_En(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);

extern void SVPWM_2Shunt_En1_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //双电阻增强 1
extern void SVPWM_2Shunt_En2_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB); //双电阻增强 2
extern void SVPWM_3Shunt_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);
extern void SVPWM_3Shunt_En_5Seg(u8 t_bObjId,PSTR_VectorAB tS_pVolAB);
/**位置: mc_declaration.h
#define Motor_FocMethod_M0 \
{ /* FOC Methods */ \
    SVPWM_3Shunt,          /* SVPWM 调制 */ \
}

```

图 9.3 SVPWM 调制方式选择

4、过调制的说明

自动过调制，最大到达母线电压的 2/3 输出。电压输出最大值大小取决于电流采样方式，双电阻时为保证可靠的采样电流，在部分扇区无法到最大输出幅值。过调制时自动实现 7 段式/5 段式调制方式切换，电压满足需要时默认采用 7 段式调制。

```

#define OVER_MODU_FACTOR_M0 ((s16)21845) /*单位: Q15, 过调制系数*/

```

图 9.4 过调制系数设置

版本历史

表-文档版本历史

时间	版本号	说明
2025.05.13	1.3.0	代码截图变更为实际的代码
2024.09.27	1.2.3	增加新平台状态机，故障保护逻辑介绍
2024.06.02	1.2.2	细化第 5 章调试步骤
2024.1.04	1.2.1	更改标题序号
2023.11.24	1.2.0	增加 VF 解耦补偿说明，增加第 9 章补充说明内容
2023.11.23	1.1.3	增加快速闭环启动，低压保护说明
2023.9.5	1.1.2	增加 07x 系列
2023.9.5	1.1.1	增加数据格式说明
2023.8.31	1.1.0	更新增加调试步骤